

VR-Algebra: Operational Tracking Discipline Applied to Algebraic Structures

Vitaly Reznik

ORCID: 0009-0002-4103-6387

2026

Version 1.0.0

Abstract

VR-Algebra is the eighth work of the cycle developing the VR programme (Reznik 2026, preprints VR. A Formal System, VR-Numbers, VR-Sets, VR-Forms, VR-Audit, VR-Sets-ZFA, VR-Apparatus). It applies the VR-Apparatus framework — the operational tracking discipline formalised as predicates carrying construction witnesses — to the standard algebraic hierarchy: additive groups, rings, multiplicative groups, fields, and modules. Five operational typeclasses are constructed as Lean 4 layers over the corresponding mathlib structures, each registering a predicate on its carrier and closure axioms for its operations. The construction reuses the VR-Apparatus identity (PredicateOperationality) without modification across five algebraic structures; in the case of modules the apparatus instance is literally inherited from the underlying operational additive group without a new registration. Substantive Mode A theorems are proved for all standard operations (closure under addition, multiplication, inverse, scalar multiplication, integer powers). One substantive Mode B audit is conducted — the image of an operational subgroup under an operational homomorphism is an operational subgroup — whose proof consists of a single inference step over the apparatus, contrasting structurally with the analytical Mode B audit of VR-Audit (Hahn-Banach). The cycle is logically constructive with two documented structural exceptions located in field-inverse reasoning; these exceptions trace to mathlib’s encoding of $\text{Inv } K$ resolution and are recorded as Finding A15/A16 with detailed reconnaissance. The recognition discipline pattern is observed in both removal (preemptive abstractions deleted) and introduction (justified abstractions added) forms, completing the bidirectional methodological signature anticipated in the VR-Apparatus formalisation. Twenty findings are catalogued (A0-A19) covering apparatus reuse, axiom hierarchy, recognition discipline, and engineering observations.

Companion Lean formalisation. The Lean 4 code is at <https://github.com/inventor1975/VRCycle>, tag `v1.12-vr-operational-algebra-v1.0.0`, commit `5e8f8e0`. 64 public objects across 8 files in the `VR-Cycle/Algebra/` directory, 3,977 lines of Lean source, full axiom audit via `#print axioms` for every public object, zero `sorry` or `admit`. Unlike the prior seven works of the VR cycle, no companion Lean Zenodo record accompanies this preprint; the Lean code is cited by GitHub tag rather than DOI, simplifying the publication workflow without compromising reproducibility.

Keywords: operational tracking discipline, predicate operationality, additive group, ring, field, module, apparatus reuse, recognition discipline, axiom hierarchy, Mode A closure theorems, Mode B audit, constructive content, Lean 4, mathlib, formal verification.

Part I. Foundations

I.1. Place in the cycle

VR-Algebra is the eighth work in the cycle developing the VR programme:

- **VR. A Formal System** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20212092) — a minimal axiomatisation of arithmetic on three primitives $\{\emptyset, \rightarrow, t\}$ and four axioms, equivalent to Peano arithmetic.
- **VR-Numbers** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20272743) — operational extensions over the natural numbers: $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$ as operational procedures, without introducing pairs as objects.
- **VR-Sets** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20303536) — an operational theory of sets: a set is a describable functionality, $\in$ is reference rather than containment. Russell’s paradox dissolves ontologically.
- **VR-Forms** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20355939) — a two-register apparatus on top of VR-Sets: ontological register (operational sets) and formal register (syntactic descriptions without ontological commitment); conservativity theorem secures safety.
- **VR-Audit** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20364111) — operational reformulation of classical analysis: the Hahn-Banach extension theorem treated as Mode B audit revealing what survives operational restriction.
- **VR-Sets-ZFA** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20369346) — coinductive parallel to VR-Sets accommodating non-well-founded sets with operational interpretation.
- **VR-Apparatus** (Reznik, 2026; Zenodo DOI 10.5281/zenodo.20381417) — abstract formalisation of the operational tracking discipline as Lean 4 typeclasses: `PredicateOperationality`, Mode A and Mode B patterns, reference apparatuses, inter-apparatus morphisms.

VR-Algebra is the present work. It applies the apparatus formalised abstractly in VR-Apparatus to the standard algebraic hierarchy. Where VR-Audit demonstrated the apparatus on analysis (Hahn-Banach as Mode B audit), VR-Algebra demonstrates the apparatus on algebra: groups, rings, fields, modules, with substantive Mode A theorems throughout and one substantive Mode B audit (image of subgroup).

The substantive connection with the preceding works: VR-Apparatus formalised the framework; VR-Algebra exercises it in a second domain (after analysis). Together with VR-Audit, the present work establishes that the apparatus is not analysis-specific — its reuse pattern generalises to algebraic structures with the same architectural simplicity.

I.2. The initial position

The slogan of the whole cycle: “only $\emptyset$ is, all else is doing”. Applied to arithmetic, it gave VR; to numbers, VR-Numbers; to sets, VR-Sets; to formal language, VR-Forms; to classical analysis (as audit), VR-Audit; to non-well-founded sets, VR-Sets-ZFA; to the apparatus itself, VR-Apparatus. In all seven prior works the slogan operated at the level of mathematical content: the entities under study are operational functionalities, and the apparatus tracks which operations preserve operationality.

VR-Algebra adds nothing to the ontology. It applies the existing apparatus to algebraic content. The novelty is not in mathematical position but in scope of demonstration: the apparatus framework is general across algebraic structures, the recognition discipline acts bidirectionally, and the axiom hierarchy of the resulting Lean formalisation has structural properties (Finding A10, Finding A15) that are visible only at this level of integration.

The full formulation of the position for VR-Algebra:

“The apparatus formalised in VR-Apparatus applies without modification to the algebraic hierarchy. Operational tracking discipline is general across mathematical domains; algebra and analysis exercise the same architectural framework with structurally different content.”

I.3. What this work demonstrates

Five substantive claims, each established with explicit Lean evidence:

(1) Apparatus generality across algebraic structures. The `PredicateOperationality` typeclass from `VR-Apparatus` registers without modification for additive groups, rings, multiplicative groups, fields, and modules. In four cases a new apparatus instance is declared (one per structure); in the fifth (modules) the apparatus instance from the underlying additive group is literally reused — no new registration is needed because modules add scalar action without introducing a new carrier-level predicate. This is the strongest confirmation of apparatus generality: from “framework applies without modification” to “framework instance already exists, no work required”.

(2) Mode A theorems form a complete closure system. Every standard algebraic operation (addition, negation, subtraction, scalar action by natural and integer numbers, multiplication, integer powers, inverse, division, scalar multiplication for modules) is provided with a Mode A theorem asserting that it preserves operationality. Eight Mode A theorems for additive operations, two for ring operations, five for multiplicative operations, one for field operations, one for module scalar action — fifteen Mode A theorems in total, all with full axiom audit. The Mode A pattern from `VR-Apparatus` admits both unary and binary heterogeneous forms; the heterogeneous case (scalar multiplication for modules) is the first in the VR cycle and refines the Mode A definition slightly.

(3) Substantive Mode B audit is structurally simpler in algebra than in analysis. The Mode B audit conducted in `VR-Algebra` is: if $\varphi : G \rightarrow H$ is an operational homomorphism and S is an operational subgroup of G , then $\varphi(S)$ is an operational subgroup of H . The proof consists of a single inference step over the apparatus, after `AddSubgroup.mem_map` supplies the existential witness. This contrasts structurally with the Mode B audit in `VR-Audit` (Hahn-Banach extension): there, the Mode B proof required multiple supporting lemmas, classical existence selection through Zorn’s lemma (or equivalent), and the constructive witness was selected from a non-empty set rather than supplied directly. The algebraic Mode B audit is constructive throughout; the analytical Mode B audit is classical with operational wrapping.

(4) Axiom ceiling is a carrier-type property, not an algebraic depth property. The Lean axiom audit for the operational typeclasses on $\mathbb{Z}$, $\mathbb{ZMod}\ n$, and $\mathbb{Q}$ yields a clear pattern: the axiom ceiling of every algebraic structure built on a given carrier type is identical to the carrier type’s intrinsic ceiling. Algebraic depth (additive group $\rightarrow$ ring $\rightarrow$ field $\rightarrow$ module) does not escalate the axiom dependency. This is observed across four levels of the hierarchy for $\mathbb{Q}$ (group, ring, field, module — all at `[propext, Classical.choice, Quot.sound]`) and for $\mathbb{Z}$ (group, ring, module — all at `[propext]`). The principle is recorded as Finding A10 and confirmed structurally.

(5) Recognition discipline acts bidirectionally. The methodological pattern anticipated in `VR-Apparatus` — that preemptive abstractions should be removed when no instances yet justify them — is applied five times in the present cycle (Findings A0, A6, A9, plus two Stage 5 v0.2.0 omissions). The reverse direction — that abstractions should be introduced when instances justify them — is exercised once: the multiplicative `OperationalGroup` typeclass, removed in v0.1.0 as preemptive (Finding A0), is reintroduced in v0.3.0 because the field-of-units construction provides a non-trivial instance (Finding A12). The pattern is further extended by a seventh application: the recognition discipline applied not to code but to the plan itself, when the heterogeneous binary scalar action operation revealed that the planned `IsModeA0p2` form was architecturally incorrect (Finding A19). The bidirectional pattern, completed across seven applications, constitutes a methodological signature of the cycle.

I.4. Connection with the preceding works

Connection with VR-Apparatus. VR-Apparatus formalised the operational tracking discipline as Lean 4 typeclasses. VR-Algebra applies that formalisation; it does not extend it. Every class `PredicateOperationality`, every `IsModeAOp`, every `IsModeBOp` from VR-Algebra is the corresponding typeclass from VR-Apparatus. The relation is exactly that between an abstract framework and a substantive application: the framework provides the apparatus identity; the application provides the content.

What VR-Algebra adds to VR-Apparatus by way of feedback: five confirmations of Finding A3 (apparatus instance pattern works across structures), one confirmation that the apparatus instance can be reused without new declaration (modules), and one extension of the Mode A definition to heterogeneous binary operations (Finding A19). None of these alters VR-Apparatus; all of them clarify its scope.

Connection with VR-Audit. VR-Audit conducted the apparatus on analysis: the Hahn-Banach theorem treated as Mode B audit. VR-Algebra conducts the apparatus on algebra. The two together demonstrate that the apparatus framework is general across mathematical domains.

The structural contrast between the analytical and algebraic Mode B audits is recorded as Finding A11. In analysis, the substantive Mode B step (extending a bounded linear functional from a subspace to the whole space) requires classical existence selection — Zorn’s lemma or its equivalent — which translates into `Classical.choice` in the Lean axiom audit. In algebra, the substantive Mode B step (showing that the image of an operational subgroup under an operational homomorphism is operational) uses a constructive witness supplied by `AddSubgroup.mem_map`, and the proof reduces to a single inference step over the apparatus. The audit in algebra is constructive; the audit in analysis is classical. This is not a defect or limitation in either case — it is a structural property of the underlying mathematics, surfaced by the apparatus tracking.

Connection with VR-Numbers. VR-Algebra instantiates its typeclasses on $\mathbb{Z}$, $\mathbb{ZMod}\ n$, and $\mathbb{Q}$ — types that are mathematically standard and whose operational status was discussed in VR-Numbers. The present work does not redefine these types; it instantiates the algebraic typeclasses on `mathlib`’s `Int`, `ZMod n`, and `Rat`, with the operational predicate set to the constant `True` (consistent with the position established in VR-Numbers that integer and rational arithmetic is operational throughout).

I.5. What is not in scope

VR-Algebra v1.0.0 is **stable release of the algebraic hierarchy core**. The following are explicitly **not** in scope of the present cycle:

Polynomial rings. The construction `OperationalRing (Polynomial R)` when `[OperationalRing R]` would extend the apparatus to polynomial structures. This is a natural future direction (anticipated as part of a possible v1.1.0+ extension) and is sketched in Part IX. It is **not** included in v1.0.0 because the algebraic hierarchy core (groups, rings, fields, modules over fields) is itself substantial enough for a coherent cycle.

Matrix rings. Similarly, `OperationalRing (Matrix n n R)` is a natural extension and is sketched in Part IX, not included in v1.0.0.

Nontrivial operational predicates. In v1.0.0, every operational typeclass instance uses `IsOperational := fun _ => True`. This is consistent with the position that $\mathbb{Z}$, $\mathbb{ZMod}\ n$, and $\mathbb{Q}$ are operational throughout (no element of these types has special operational status; the predicate would distinguish nothing). The first nontrivial operational predicate would arise on a domain such as the ring of computable functions $\mathbb{R} \rightarrow \mathbb{R}$, where computability genuinely distinguishes a subring; or on the ring of algebraic integers, where integrality genuinely distinguishes a subring. Such constructions are anticipated in Part IX as a possible future direction.

Additional substantive Mode B audits. The present cycle conducts one substantive Mode B audit (image of operational subgroup). The pattern is established. Further audits — Lagrange’s theorem, Sylow theorems, fundamental theorem of finitely generated abelian groups — would each be substantial standalone investigations, anticipated in Part IX.

Multiplicative group instances beyond fields. The multiplicative `OperationalGroup` typeclass is introduced in v0.3.0 with a single substantive instance: the group of units of an operational field. Other potential sources of multiplicative group instances (Galois groups, permutation groups, etc.) are not included.

Topology, category theory, other domains. These would be separate cycles, parallel to VR-Algebra but on different mathematical content. Brief discussion is given in Part IX.

I.6. Structure of the work

The work consists of nine parts:

Part I. Foundations. Place in the cycle, position, what is demonstrated, what is not in scope, structure of the work.

Part II. Operational typeclasses. Definitions of `OperationalAddGroup`, `OperationalRing`, `OperationalGroup`, `OperationalField`, `OperationalModule`. Closure axioms, predicate registration, bridge instances between typeclasses. Methodological discussion of when to introduce a new predicate (Field) vs reuse (Module).

Part III. Apparatus reuse. Finding A3 narrative across five algebraic structures. Module case as strongest confirmation. Methodological implications for apparatus generality.

Part IV. Mode A theorems. Complete enumeration of closure theorems across the algebraic hierarchy. Additive (8), ring (2), multiplicative (5), field (1), module (1). Discussion of heterogeneous binary operations (Finding A19).

Part V. Mode B audit. The image-of-subgroup theorem. One-step proof structure. Constructive witness through `AddSubgroup.mem_map`. Structural contrast with VR-Audit’s Hahn-Banach (Finding A11).

Part VI. Recognition discipline. Bidirectional pattern: five removal applications, one introduction application, one plan-level correction. Seven applications complete the methodological signature.

Part VII. Axiom analysis. Three-tier hierarchy. Finding A10 (ceiling = carrier property). Finding A15/A16 (structural `Classical.choice` in field-inverse reasoning). Honest constructivity framing.

Part VIII. Lean formalisation methodological observations. Analogous to VR-Forms Part IX and VR-Sets Part X. Twenty findings (A0-A19) catalogued by category.

Part IX. Open questions and future directions. Polynomial rings, matrix rings, nontrivial operational predicates, additional substantive Mode B audits, multiplicative group instances beyond fields, possible operational cycles in other domains.

Part II. Operational Typeclasses

II.0. Introduction

Part I positioned the work: the apparatus formalised in VR-Apparatus is applied to the standard algebraic hierarchy. Part II constructs the application.

The construction consists of five operational typeclasses — one per algebraic structure — each layered over the corresponding mathlib structure. Each typeclass:

- registers a predicate on the carrier type, identifying which elements are operational;
- carries closure axioms asserting that the structure’s operations preserve operability;
- registers the carrier as a `PredicateOperability` instance (the apparatus identity from VR-Apparatus);
- where applicable, provides a bridge instance to an underlying operational typeclass, so that the algebraic hierarchy is also a hierarchy of operational typeclasses.

The five typeclasses are:

- `OperationalAddGroup G` — for `AddGroup G`;
- `OperationalRing R` — for `Ring R`, extending `OperationalAddGroup R`;
- `OperationalGroup G` — for `Group G` (the multiplicative case);
- `OperationalField K` — for `Field K`, extending both `OperationalRing K` and `OperationalGroup K`;
- `OperationalModule R M` — for `Module R M`, requiring `[OperationalRing R]` and `[OperationalAddGroup M]` separately.

The structural pattern of the construction is uniform: predicate, closure axioms, apparatus instance, bridges. The architectural variation — when to introduce a new predicate vs reuse one inherited from a bridge — is the principal point of methodological interest. Modules in particular reuse the predicate from the underlying additive group without introducing a new one; this is taken up in §II.5.

In every instance presented in §§II.1–II.5, the operational predicate is set to the constant `True`. This is consistent with the position from VR-Numbers: integer and rational arithmetic is operational throughout; no element of $\mathbb{Z}$, $\mathbb{Z}/n$, or $\mathbb{Q}$ has special operational status that the predicate could distinguish. The apparatus framework itself is general — the predicate may in principle distinguish a meaningful subset (computable functions, algebraic integers, decidable subgroups, etc.) — but no such instance is constructed in the present work. Possible nontrivial-predicate instances are discussed in Part IX.

The chapter contents: §II.1 develops `OperationalAddGroup` in full detail, establishing the pattern that subsequent typeclasses reuse. §II.2 develops `OperationalRing` with the bridge to additive groups. §II.3 develops the multiplicative `OperationalGroup`. §II.4 develops `OperationalField` with the inverse-closure axiom and the bridge to multiplicative groups via the units construction. §II.5 develops `OperationalModule` in the bridge-based form, with explicit comparison to the predicate-introducing typeclasses. §II.6 collects the comparative discussion: when to introduce a new predicate, and when to reuse one inherited from a bridge. §II.7 summarises the part.

II.1. OperationalAddGroup

Position

The first operational typeclass in the algebraic hierarchy is `OperationalAddGroup`, layered over `mathlib`'s `AddGroup`. An additive group is a type `G` with a binary operation `+`, a unary operation `-` (negation), a zero element `0`, and a binary operation `-` (subtraction) defined as $a - b = a + (-b)$, all satisfying the group axioms.

An operational additive group is an additive group equipped with:

- a predicate `IsOperational : G → Prop`, identifying which elements are operational;
- closure axioms asserting that the operations `+`, `-`, `-` (subtraction), and the element `0` preserve operat-
ionality.

The motivation is direct: in an operational context, one wants to know that constructing an element by group operations from operational starting elements yields an operational result. The apparatus framework from VR-Apparatus provides the typeclass shape; `OperationalAddGroup` instantiates it for additive groups.

Closure axioms

The closure axioms of `OperationalAddGroup` are:

- `zero_isOperational : IsOperational (0 : G)` — the zero element is operational.
- `add_isOperational : ∀ a b, IsOperational a → IsOperational b → IsOperational (a + b)` — addition preserves operat-
ionality on operational inputs.
- `neg_isOperational : ∀ a, IsOperational a → IsOperational (-a)` — negation preserves
operat-
ionality.
- `sub_isOperational : ∀ a b, IsOperational a → IsOperational b → IsOperational (a - b)` — subtraction preserves operat-
ionality.

Subtraction is included explicitly rather than derived from addition and negation. This choice is method-
ological: closure axioms are stated for every operation provided by `AddGroup`, so that downstream
proofs can invoke the relevant closure axiom directly without first unfolding subtraction. The derivation
`sub_isOperational := fun a b ha hb => add_isOperational a (-b) ha (neg_isOperational b
hb)` is admissible and may be used in concrete instances, but at the typeclass level the axiom is independent.

The zero element is operational unconditionally. This reflects the structural role of zero in any additive group:
zero is determined by the group structure itself, not constructed from other elements. The apparatus tracks
construction from operational inputs; zero is the operational base from which other operational elements may
be reached by the operations.

Apparatus instance

The apparatus framework of VR-Apparatus provides a typeclass `PredicateOperationality` parameterised
on a type and a predicate. To register `G` as an apparatus instance, one provides:

```
instance instPredOpAddGroup : PredicateOperationality G IsOperational
```

For `OperationalAddGroup G`, this instance is constructed by setting the `IsOperational` field of `Predica-
teOperationality` to the `IsOperational` field of `OperationalAddGroup`. The instance is a direct identifi-
cation: the apparatus tracks the same predicate that the algebraic typeclass declares.

This identification is the principal architectural feature: the operational typeclass on the algebraic structure **is** the apparatus instance on the carrier with the same predicate. No translation is needed between the algebraic and apparatus levels.

Subgroup predicate

Subgroups of an operational additive group inherit a natural operational predicate. The subgroup $S \subseteq G$ is operational if every element of S is operational. Equivalently: the predicate on S is the restriction of `IsOperational` to the subgroup carrier.

In the Lean formalisation this is a separate predicate `IsOperationalSubgroup S` defined on `AddSubgroup G`. The predicate is not bundled into a structure: there is no `OperationalAddSubgroup` typeclass with constructors and instances. The reason — the recognition discipline at work — is that no use case had emerged at the time of v0.1.0 that required polymorphic subgroup handling beyond the predicate. A bundled structure would have added abstraction without instances justifying it; the predicate suffices. This was recorded as Finding A6 (recognition discipline removal) and is taken up in Part VI.

The basic results on operational subgroups are:

- `bot_isOperationalSubgroup : IsOperationalSubgroup (⊥ : AddSubgroup G)` — the trivial subgroup $\{0\}$ is operational unconditionally.
- `top_isOperationalSubgroup` — the full subgroup G is operational conditionally on every element of G being operational, which is automatic when the operational predicate is `True`.
- Intersection results: the intersection of two operational subgroups is operational; bilateral intersection over a family follows by induction.

These are foundational results, supporting the substantive Mode B audit in Part V.

Instances

The operational additive group typeclass admits the following instances in v1.0.0:

- `instOperationalAddGroupInt : OperationalAddGroup  $\mathbb{Z}$` — with `IsOperational := fun _ => True` and trivial closure axioms.
- `instOperationalAddGroupZMod : OperationalAddGroup (ZMod n)` — with `IsOperational := fun _ => True` and trivial closure axioms.
- The rational case is provided as a bridge from `OperationalField  $\mathbb{Q}$` (see §II.4) — `OperationalAddGroup  $\mathbb{Q}$` is derived rather than declared.

The integer and `ZMod n` instances are direct: every integer is operational, every residue class modulo n is operational. The triviality of the predicate reflects the position established in VR-Numbers — integer arithmetic is operational throughout, $\mathbb{Z}$ as a constructive completion of $\mathbb{N}$ has no operational distinction at the level of individual elements.

Mode A theorems

The closure axioms of `OperationalAddGroup` directly entail Mode A theorems in the sense of VR-Apparatus: each operation, viewed as a function on the carrier, is Mode A with respect to the operational predicate. The Mode A theorems for the additive structure are:

- `add_isModeAOp` — addition is Mode A binary;
- `neg_isModeAOp` — negation is Mode A unary;

- `sub_isModeAOp` — subtraction is Mode A binary;
- `nsmul_isOperational` — natural-number scalar multiplication on operational elements yields operational elements (proven by induction on $\mathbb{N}$);
- `zsmul_isOperational` — integer scalar multiplication likewise (proven by case analysis on the sign of the integer).

The `nsmul` and `zsmul` theorems are not stated as `IsModeAOp` typeclass instances directly because the scalar argument is of a different type ($\mathbb{N}$ or $\mathbb{Z}$) than the carrier (G). Instead they are stated as closure properties — for each scalar, the corresponding map $G \rightarrow G$ preserves operability on operational elements. This is the unary form, applied to the family of maps parameterised by the scalar. The full Mode A treatment of heterogeneous binary operations is developed for module scalar multiplication in §II.5 and Part IV.

The full collection of Mode A theorems on additive operations is given in Part IV.

Position established

The structure of `OperationalAddGroup` — predicate, closure axioms, apparatus instance, subgroup predicate, Mode A theorems — is the architectural template for the remaining four operational typeclasses. The variations across the typeclasses lie in which closure axioms are needed, whether a bridge to an underlying typeclass is provided, and whether a new predicate is introduced or one inherited from a bridge is reused. §§II.2–II.5 develop these variations on the base pattern.

II.2. OperationalRing

Position

A ring is an additive group R equipped with multiplication $\cdot$ and a multiplicative identity 1 , satisfying distributivity and associativity. An operational ring is a ring whose additive structure is operational and whose multiplicative structure also preserves operability.

The architectural decision: `OperationalRing R` introduces a new predicate on R (denoted `OperationalRing.IsOperational` to distinguish from `OperationalAddGroup.IsOperational`), even though the carrier is the same R . The reason is semantic: ring operations introduce multiplicative content that is not present in the additive structure. The predicate of the operational ring is, in principle, distinct from the predicate of the operational additive group — though in current instances both are `True` and coincide extensionally.

This is the first architectural decision where introducing a new predicate is the right choice. The contrast case — `OperationalModule`, which introduces no new predicate — is developed in §II.5.

Closure axioms

The closure axioms of `OperationalRing` are:

- `one_isOperational` : `IsOperational (1 : R)` — the multiplicative identity is operational.
- `mul_isOperational` : $\forall a\ b, \text{IsOperational } a \rightarrow \text{IsOperational } b \rightarrow \text{IsOperational } (a * b)$ — multiplication preserves operability.

The closure axioms for the additive operations (`add`, `neg`, `sub`, `zero`) are not repeated in `OperationalRing`; they are provided through the bridge instance to `OperationalAddGroup` (see below). The ring typeclass adds only the new content: multiplicative identity and multiplication.

Natural-number powers (`npow`) are derivable: $a^0 = 1$ and $a^{(n+1)} = a * a^n$ reduces to the multiplicative closure axiom. The Mode A theorem `npow_isOperational` is proved by induction in Part IV; it is not a closure axiom.

Bridge instance

The architectural feature distinguishing `OperationalRing` from `OperationalAddGroup` is the bridge:

```
instance toOperationalAddGroup [OperationalRing R] : OperationalAddGroup R
```

This instance says: every operational ring is an operational additive group, with predicate identification. The bridge is constructed by inheriting `IsOperational` from the ring instance and providing the additive closure axioms via standard ring lemmas. The new ring predicate becomes the additive group predicate; closure under addition follows from closure under multiplication via the identity $a + b = -(-a - b)$ if needed, or directly when the ring instance is constructed.

The bridge has structural consequences. When `[OperationalRing R]` is in scope, the operational additive group structure on R is automatically available. Theorems proved for operational additive groups (e.g., subgroup results from §II.1) apply to the additive structure of every operational ring. The Lean typeclass search handles the chain transparently.

Apparatus instance

The ring case introduces its own apparatus instance:

```
instance instPredOpRing : PredicateOperationality R IsOperational
```

This instance is parallel to `instPredOpAddGroup` but on the ring predicate. The duplication is unavoidable: the additive group apparatus instance tracks the additive group predicate; the ring apparatus instance tracks the ring predicate. The two predicates are extensionally equal in current instances (both `True`) but typeclass-level distinct.

Instances

The operational ring typeclass admits the following instances:

- `instOperationalRingInt` : `OperationalRing ℤ` — integers as operational ring.
- `instOperationalRingZMod` : `OperationalRing (ZMod n)` — residue class rings as operational rings.
- `instOperationalRingRat` : `OperationalRing ℚ` — derived from `OperationalField ℚ` (see §II.4), not declared directly.

For $ℤ$ and $ZMod\ n$, the operational ring instance is the natural one. The corresponding operational additive group instances (from §II.1) are inherited from the ring instances via the bridge, eliminating any potential diamond conflict.

Mode A theorems

The Mode A theorems for ring operations are:

- `mul_isModeAOp` — multiplication is Mode A binary.
- `npow_isOperational` — natural-number powers preserve operability.

`one_isOperational` is a constant rather than a Mode A theorem; it is asserted as a closure axiom in the typeclass.

Full enumeration of the Mode A theorems is in Part IV.

A note on commutativity

A natural extension would be `OperationalCommRing R`, layered over `mathlib's CommRing`. This typeclass would add commutativity to the ring structure without introducing new closure axioms.

The recognition discipline applies. At the level of operational tracking, commutativity adds no new content: closure under multiplication is the same axiom whether the ring is commutative or not. A separate `OperationalCommRing` typeclass would multiply abstractions without providing new structural content. It is omitted from v1.0.0. This decision is recorded as Finding A9 and discussed in Part VI.

If a future use case emerges where commutativity is needed at the operational level — for instance, to state that polynomial rings over operational commutative rings are themselves operational rings — the typeclass can be introduced then. Until such a use case appears, the abstraction is not justified.

II.3. OperationalGroup

Position

The multiplicative group case is structurally parallel to the additive group case. `OperationalGroup G` is layered over `mathlib's Group G`, with predicate, closure axioms for multiplication, inverse, division, and unconditional operability of the multiplicative identity.

Closure axioms

The closure axioms of `OperationalGroup` are:

- `one_isOperational` : `IsOperational (1 : G)` — multiplicative identity is operational.
- `mul_isOperational` : $\forall a\ b, \text{IsOperational } a \rightarrow \text{IsOperational } b \rightarrow \text{IsOperational } (a * b)$.
- `inv_isOperational` : $\forall a, \text{IsOperational } a \rightarrow \text{IsOperational } (a^{-1})$.
- `div_isOperational` : $\forall a\ b, \text{IsOperational } a \rightarrow \text{IsOperational } b \rightarrow \text{IsOperational } (a / b)$.

The structural parallel to `OperationalAddGroup` is exact, with $+$ $\leftrightarrow$ $*$, $-$ $\leftrightarrow$ $^{-1}$, $-$ (subtraction) $\leftrightarrow$ $/$ (division), $0 \leftrightarrow 1$. The closure axioms mirror precisely.

Mode A theorems

The Mode A theorems for the multiplicative group are:

- `mul_isModeAOp`, `inv_isModeAOp`, `div_isModeAOp` — for the basic operations.
- `npow_isOperational` — natural-number multiplicative power preserves operability (by induction).
- `zpow_isOperational` — integer multiplicative power preserves operability (by case analysis on the sign).

The proof of `zpow_isOperational` differs slightly in mechanics from `zsmul_isOperational` (its additive counterpart), even though the structural pattern is the same: in `zpow`, the case analysis on `Int.ofNat n`

admits a bare exact while `zsmul` requires an explicit rewrite step. This is an engineering observation about `mathlib`'s `zsmul` definitional unfolding versus `zpow` (Finding A18); the mathematical content is the same.

A note on the development history

The multiplicative `OperationalGroup` typeclass was initially omitted from the cycle. At the point when only additive operations and additive instances were under construction (the start of the cycle), no instances of multiplicative groups had been identified that could not be handled directly through ring or field machinery. By the recognition discipline, the typeclass was not introduced preemptively — the abstraction was not justified by any concrete instance. This omission is recorded as Finding A0.

When the field-of-units construction was developed (§II.4 below), it became natural to declare the units of an operational field as an operational multiplicative group. The instance is direct: `OperationalGroup K^×` for any `OperationalField K`. This provided the first non-trivial instance of `OperationalGroup`, retroactively justifying the introduction of the typeclass. The typeclass was added in v0.3.0, closing the recognition discipline loop (removal followed by justified introduction). This is recorded as Finding A12 and discussed further in Part VI.

The episode is the principal illustration of bidirectional recognition discipline: abstractions are removed when not justified (A0), and reintroduced when subsequently justified (A12). The pattern is not “remove all abstractions on principle” — it is “track justification, both for removal and for introduction”.

II.4. OperationalField

Position

A field is a commutative ring K in which every nonzero element has a multiplicative inverse. The operational structure on a field combines the operational ring structure with operational inverse — but the inverse, unlike the additive group's negation, is partial: defined only on nonzero elements, with conventional value $0^{-1} = 0$ in `mathlib`'s encoding.

`OperationalField K` is layered over `mathlib`'s `Field K`, with closure axioms for ring operations (inherited through bridge to `OperationalRing K`) and an additional closure axiom for inverse. The architectural decision: `OperationalField K` introduces a new predicate — distinct from `OperationalRing.IsOperational` — because inverse is a genuinely new operation that is not present in the ring structure.

This is a deliberate design choice. Alternative architectures could have:

- Reused the ring predicate and added only the inverse closure axiom. This would suffice mathematically: in current instances all predicates are `True`. But typeclass-level it would conflate the “operational under ring operations” and “operational under field operations” distinctions, which the framework keeps separate.
- Introduced inverse closure as a hypothesis on the typeclass without a new predicate. This is possible but less uniform — every other operational typeclass owns its predicate.

The chosen architecture — new predicate for the field — keeps the structural pattern uniform across the typeclasses that introduce new mathematical content. The contrast case is `OperationalModule`, which does not introduce a new predicate because it adds an action, not a carrier-level operation. The architectural distinction — predicate for new carrier structure, reuse for action only — is the principal methodological insight of §II.5.

Closure axioms

The closure axioms of `OperationalField` are:

- `zero_isOperational : IsOperational (0 : K).`
- `one_isOperational : IsOperational (1 : K).`
- `add_isOperational, neg_isOperational, sub_isOperational` — additive closure.
- `mul_isOperational` — multiplicative closure.
- `inv_isOperational : ∀ a, IsOperational a → IsOperational a-1` — inverse closure.

The redundancy with `OperationalRing` axioms is structural: every operational field is an operational ring (via the bridge below), so the additive and ring closure axioms appear in both typeclasses. The Lean instance construction provides them directly; downstream code accesses them through whichever typeclass is in scope.

The inverse closure axiom is unconditional: `IsOperational a → IsOperational a-1`, without the requirement that $a \neq 0$. In `mathlib`'s encoding, $0^{-1} = 0$, so inverse is total. Closure on the zero case is automatic (zero is operational, inverse of zero is zero which is operational). The axiom thus subsumes both the nonzero case (genuine inverse) and the convention case (zero inverse).

Bridge instances

The field case has two bridge instances:

```
instance toOperationalRing [OperationalField K] : OperationalRing K
instance toOperationalGroupUnits [OperationalField K] : OperationalGroup K×
```

The first bridge — to operational ring — is direct: every operational field is an operational ring, with the field predicate inherited as the ring predicate, and the ring closure axioms inherited from the corresponding field axioms.

The second bridge — to operational multiplicative group on the units — is more substantive. The group of units $K^\times$ of a field K consists of nonzero elements (since every nonzero element has an inverse). For an operational field, the units inherit operability from the field: a unit $u : K^\times$ is operational if its underlying field element is operational. The bridge constructs an `OperationalGroup` instance on $K^\times$ from an `OperationalField` K instance, with the appropriate transfer of predicate and closure axioms.

This second bridge is the substantive instance closing the recognition discipline loop (Finding A12): the multiplicative `OperationalGroup`, omitted in v0.1.0 for lack of instances, is reintroduced in v0.3.0 because the field-of-units construction provides a non-trivial instance.

Inverse Mode A theorem and the field structure

The Mode A theorem `inv_isModeAOp_field` — inverse on a field is Mode A unary — uses the inverse closure axiom directly. The mathematical content is one line.

However, the axiom audit on this theorem and on `toOperationalGroupUnits` reveals a structural property of `mathlib`: both objects, in spite of constructive content, are flagged as depending on `Classical.choice` in the axiom audit. The cause is structural and is documented in Part VII as Finding A15/A16.

Briefly: `mathlib`'s encoding of `Inv K` for a generic `Field K` resolves through infrastructure that includes `Mathlib.Data.Real.Basic`, which in turn introduces real-number instances that propagate `Classical.choice` to anything reasoning about field-level inverse. The Lean axiom audit registers this dependency. A systematic reconnaissance (six test files, documented in Part VII) confirmed that no isolation

of the field-inverse objects avoids the propagation: it is structural to mathlib’s encoding, not an artefact of the cycle’s implementation.

The result is recorded honestly: the cycle is logically constructive — no closure axiom or Mode A theorem requires `Classical.choice` in its mathematical content — but mathlib’s encoding of field inverse introduces a structural `Classical.choice` dependency in the proof of `inv_isModeAOp_field` and `toOperationalGroupUnits`. This is two of sixty-four public objects in the cycle, both in field-inverse reasoning. The propagation extends to $\mathbb{Q}$ -Module objects via the bridge chain (§II.5).

Instances

The operational field typeclass admits one substantive instance in v1.0.0:

- `instOperationalFieldRat : OperationalField  $\mathbb{Q}$` — the rationals as operational field.

Other potential field instances (algebraic number fields, finite fields beyond `ZMod p` for prime p) are not provided in v1.0.0 but are not excluded in principle.

Axiom hierarchy

`instOperationalFieldRat` has axiom profile `[propext, Classical.choice, Quot.sound]` — the full ceiling. The `Classical.choice` enters through `Rat.instField` in mathlib, propagated through the chain of bridge instances. This is consistent with Finding A10: the axiom ceiling of every algebraic structure on $\mathbb{Q}$ is `[propext, Classical.choice, Quot.sound]`, determined by $\mathbb{Q}$ as the carrier type, not by the algebraic depth of the structure built on it.

II.5. OperationalModule

Position

A module over a ring R is an abelian group M equipped with a scalar action $R \rightarrow M \rightarrow M$ satisfying distributivity and associativity with respect to ring multiplication. An operational module combines an operational ring R and an operational additive group M with a closure axiom for the scalar action.

The architectural decision: `OperationalModule R M` does **not** introduce a new predicate. The predicate for M -elements is `OperationalAddGroup.IsOperational (G := M)`, inherited from the underlying operational additive group. The predicate for R -elements is `OperationalRing.IsOperational (R := R)`, inherited from the underlying operational ring. `OperationalModule` adds only the scalar-action closure axiom.

This is the contrast case to `OperationalField`. The reasoning is direct: modules introduce an action of R on M , not a new operation on the carrier M . The set of operational elements of M is already determined by `[OperationalAddGroup M]`; the scalar action does not create new elements, only acts on existing ones. A new predicate on M would be redundant: it would either coincide with the additive group predicate (the trivial case) or contradict it (the structurally impossible case).

The structural distinction:

- **Field:** introduces inverse, a new operation on the carrier K . New predicate is natural — operability under field operations may, in principle, distinguish a subset of operational ring elements.
- **Module:** introduces scalar action, not a new operation on the carrier M . The scalar action takes operational scalars and operational module elements to operational module elements; no new operational status arises on M .

This contrast — predicate for new carrier structure, reuse for action only — is the principal methodological observation of §II.6 and is recorded as Finding A17 in the catalog.

The class definition

`OperationalModule R M` is declared with the typeclass dependencies:

```
class OperationalModule (R : Type*) (M : Type*)
  [Ring R] [AddCommGroup M] [Module R M]
  [OperationalRing R] [OperationalAddGroup M] where
  smul_isOperational : ∀ {r : R} {m : M},
    OperationalRing.IsOperational r →
    OperationalAddGroup.IsOperational m →
    OperationalAddGroup.IsOperational (r • m)
```

The closure axiom has only one new content: scalar action preserves operability. Operational status of R is checked against the ring predicate; operational status of M is checked against the additive group predicate; the conclusion is operational status of the product under the additive group predicate.

This is the first heterogeneous closure axiom in the cycle. The input types R and M are distinct; the output type is M . The Mode A theorem corresponding to this axiom is unary on M , for a fixed operational scalar $r : R$ (see Part IV); the structural form is not `IsModeAOp2` (which is homogeneous binary).

Apparatus reuse

The apparatus instance for `OperationalModule R M` is **not** new: it is the existing `instPredOpAddGroup` from `OperationalAddGroup M`. No declaration is required:

```
-- no new instance needed; instPredOpAddGroup already provides
-- PredicateOperability M OperationalAddGroup.IsOperational
```

This is the strongest confirmation of apparatus reuse — Finding A17 in the catalog. In the prior four type-classes (`OperationalAddGroup`, `OperationalRing`, `OperationalGroup`, `OperationalField`), a new apparatus instance is declared, even when the predicate is unchanged from a bridge. For modules, the predicate **and** the apparatus instance are inherited without modification.

The result: the apparatus framework adapts to the module structure with no work at all. The framework instance already exists; the typeclass adds only the closure axiom. This is the limit case of “apparatus generality”: not just “framework applies without modification” but “framework instance is reused without declaration”.

Instances

The operational module typeclass admits two instances in v1.0.0:

- `instOperationalModuleIntInt : OperationalModule ℤ ℤ` — integers as $\mathbb{Z}$ -module over themselves.
- `instOperationalModuleRatRat : OperationalModule ℚ ℚ` — rationals as $\mathbb{Q}$ -module over themselves.

Both instances have trivial `smul_isOperational := fun _ _ => trivial`, as the predicates are `True` throughout. Diamond resolution on $\mathbb{Z} \text{ — } \text{OperationalModule } \mathbb{Z} \ \mathbb{Z}$ requires `[OperationalRing ℤ]` and

[`OperationalAddGroup ℤ`], both of which can be reached either directly or through bridge instances from the ring — is handled automatically by Lean’s typeclass search; no priority annotation is needed.

Axiom hierarchy

The axiom profiles of the module instances follow Finding A10 precisely:

- `instOperationalModuleIntInt` has axiom profile [`propext`] — matching $\mathbb{Z}$.
- `instOperationalModuleRatRat` has axiom profile [`propext`, `Classical.choice`, `Quot.sound`] — matching $\mathbb{Q}$.

The module structure does not escalate the carrier ceiling. The $\mathbb{Q}$ module instance inherits `Classical.choice` from `instOperationalFieldRat` via the bridge chain `Field` $\rightarrow$ `Ring` $\rightarrow$ `AddGroup`. This is consistent with Finding A15 propagation: the structural `Classical.choice` in field-inverse reasoning propagates downstream through every typeclass instance reachable from the field instance.

The four-level confirmation of Finding A10 across $\mathbb{Q}$ (group, ring, field, module) and three-level confirmation across $\mathbb{Z}$ (group, ring, module) establishes the principle structurally: the ceiling is a property of the carrier type, not of the algebraic depth.

II.6. Comparative discussion: when to introduce a new predicate

The five typeclasses developed in §§II.1–II.5 admit a clean architectural classification by the predicate-introduction question:

New predicate introduced — typeclasses where mathematical content adds a new operation on the carrier:

- `OperationalAddGroup` — introduces additive group structure; the predicate identifies operationality under additive operations.
- `OperationalRing` — introduces multiplication on the additive group’s carrier; the predicate identifies operationality under ring operations.
- `OperationalGroup` — introduces multiplicative group structure; the predicate identifies operationality under multiplicative operations.
- `OperationalField` — introduces inverse on a ring’s carrier; the predicate identifies operationality under field operations including inverse.

Predicate reused — typeclasses where mathematical content adds an action on existing carrier:

- `OperationalModule` — introduces scalar action of R on M ; the predicate on M is unchanged, since the scalar action does not produce new elements but only acts on existing ones.

The classification distinguishes **structural extension** (new operation on the carrier) from **action extension** (new operation involving the carrier but not changing its operational set). In structural extension, a new predicate is the natural design choice — the new operation may, in principle, distinguish a subset of elements that were operational under the old operations but not under the new (e.g., elements that are operational under ring operations but not under inverse). In action extension, the predicate is unchanged, because the action does not modify the carrier’s operational set.

The principle: **predicate for new carrier structure, reuse for action only**. The principle is observed in v1.0.0 across the five typeclasses; it generalises to hypothetical extensions:

- A future `OperationalRingHom` typeclass (operational ring homomorphisms) would reuse predicates from source and target rings — homomorphisms are actions on rings, not carrier-level structure.

- A future `OperationalSubring R` typeclass (operational subring structure) would reuse the ring predicate restricted to the subring carrier — subring structure restricts the operational set, doesn't introduce new content.
- A future `OperationalAlgebra R A` typeclass (operational algebra over an operational ring) would, by analogy, reuse the underlying ring predicate on `A`, since algebra structure is ring structure plus an action of `R`.

The principle is recorded as a methodological observation supporting Finding A17: apparatus reuse is structural; the design decision to introduce or reuse a predicate is structural; and the framework adapts uniformly.

II.7. Summary of Part II

Five operational typeclasses are constructed, layered over mathlib's algebraic typeclasses (`AddGroup`, `Ring`, `Group`, `Field`, `Module`):

Typeclass	Bridge to	New predicate	Apparatus instance
<code>OperationalAddGroup</code>	—	yes	new
<code>OperationalRing</code>	<code>OperationalAddGroup</code>	yes	new
<code>OperationalGroup</code>	—	yes	new
<code>OperationalField</code>	<code>OperationalRing</code> , <code>OperationalGroup</code> (units)	yes	new
<code>OperationalModule</code>	<code>OperationalRing</code> , <code>OperationalAddGroup</code>	no	reused

Each typeclass carries closure axioms for its operations. The bridge instances connect the operational typeclasses, allowing downstream code to access the appropriate operational structure when an instance is in scope. The Lean typeclass search handles the bridge chain transparently.

The principal methodological observation: the architectural decision to introduce a new predicate or reuse one inherited from a bridge tracks the distinction between **structural extension** (new operation on the carrier) and **action extension** (new operation involving the carrier without changing its operational set). The principle is observed across all five typeclasses and supports the apparatus reuse pattern catalogued as Finding A17.

The complete enumeration of Mode A theorems entailed by the closure axioms is the subject of Part IV. The substantive Mode B audit conducted within this typeclass infrastructure is the subject of Part V. The methodological observations from the construction — recognition discipline applications, apparatus reuse confirmations, axiom hierarchy properties — are catalogued through Parts VI–VIII.

Part III. Apparatus Reuse

III.0. Introduction

Part II constructed five operational typeclasses. Part III addresses a methodological observation arising from the construction: the apparatus framework from VR-Apparatus — specifically, the `PredicateOperationality` typeclass — was reused without modification across all five typeclasses, and in one case (modules) was reused without even a new declaration.

This is Finding A3 of the cycle, formulated initially in v0.1.0 and strengthened progressively through subsequent cycles. The present part documents the five confirmations and discusses the methodological implication: the apparatus framework is general across algebraic structures.

Contents: III.1 states Finding A3 in its general form; III.2 narrates the five confirmations; III.3 develops the Module case as the strongest confirmation; III.4 discusses the methodological implication for apparatus generality.

III.1. Finding A3 — statement

The apparatus framework of VR-Apparatus introduces the typeclass `PredicateOperationality`, parameterised on a type `T` and a predicate `P : T → Prop`. An instance `PredicateOperationality T P` declares that `T` carries operational tracking with respect to `P`. The instance is the apparatus identity: with it, the apparatus machinery (Mode A, Mode B, reference apparatuses) becomes applicable to the typed predicate.

Finding A3, in its general form:

The `PredicateOperationality` typeclass admits an instance for every operational structure built over a `mathlib` algebraic typeclass, without modification of its definition or interface. The instance is constructed by identifying the operational predicate of the algebraic typeclass with the predicate field of `PredicateOperationality`; no additional machinery is required.

The finding states apparatus generality across structures. The substantive content is empirical — the framework was constructed in VR-Apparatus, and its applicability across structures is demonstrated by direct instantiation. The five confirmations in §III.2 are the demonstration.

III.2. Five confirmations across structures

The five operational typeclasses developed in Part II each register a `PredicateOperationality` instance. The instances are, in source-file order:

Confirmation 1 — `OperationalAddGroup` (v0.1.0):

```
instance instPredOpAddGroup : PredicateOperationality G IsOperational
```

A new instance is declared, with the `IsOperational` field set to the predicate of `OperationalAddGroup`. The construction is direct.

Confirmation 2 — `OperationalRing` (v0.2.0):

```
instance instPredOpRing : PredicateOperationality R IsOperational
```

A new instance is declared on the ring carrier `R`, with the ring predicate. The construction mirrors the additive group case structurally.

Confirmation 3 — `OperationalGroup` (v0.3.0):

```
instance instPredOpMulGroup : PredicateOperationality G IsOperational
```

A new instance is declared on the multiplicative group carrier, with the multiplicative group predicate. The construction is parallel to the additive group case, mutatis mutandis (additive → multiplicative, `+` → `*`, etc.).

Confirmation 4 — `OperationalField` (v0.3.0):

```
instance instPredOpField : PredicateOperationality K IsOperational
```

A new instance is declared on the field carrier, with the field predicate. The construction is direct.

Confirmation 5 — OperationalModule (v0.4.0):

```
-- no new instance declared
-- instPredOpAddGroup (Confirmation 1) is reused
```

No new instance is declared. The apparatus instance on M is the one already provided by `[OperationalAddGroup M]` — specifically, `instPredOpAddGroup` from Confirmation 1. This is the qualitatively strongest confirmation, treated separately in §III.3.

In each of Confirmations 1–4, the new apparatus instance is a one-line declaration setting the `IsOperational` field of `PredicateOperationality` to the operational predicate of the new algebraic typeclass. The fields of `PredicateOperationality` (predicate, optional auxiliary apparatus components) are filled by the algebraic typeclass without translation, simulation, or adaptation. The framework definition does not change between confirmations.

In Confirmation 5, the apparatus instance is reused literally: the typeclass dependency `[OperationalAddGroup M]` already provides `instPredOpAddGroup` on M , and `OperationalModule R M` does not need to declare anything for the apparatus to become applicable on M . The downstream `Mode A` theorem `smul_isModeAOp` (Part IV) uses the existing apparatus instance directly.

III.3. The Module case — strongest confirmation

In Confirmations 1–4, the apparatus framework applies to the new typeclass with no modification. The new typeclass adds new content (closure axioms, possibly bridges), and the apparatus instance is declared as part of that content. The pattern is: apparatus framework + new algebraic content = new apparatus instance.

In Confirmation 5, the apparatus framework applies to the new typeclass with no modification **and** the new typeclass adds no new apparatus content. The new typeclass adds closure axioms for the scalar action, which are typeclass-level data; but it does not introduce a new predicate on the module carrier M (because the operational set of M is determined by `[OperationalAddGroup M]` — see §II.5 for the architectural reasoning). Consequently, no new apparatus instance is needed: the apparatus instance from `[OperationalAddGroup M]` is reused literally.

The pattern in Confirmation 5: apparatus framework + new algebraic action = no new apparatus instance, apparatus instance reused.

This is qualitatively stronger than apparatus generality without modification. It is apparatus reuse without redeclaration. The framework does not merely admit a new instance for each new typeclass; it admits the same instance, used in a new typeclass context.

The structural reason has been articulated in §II.5: the architectural distinction between **structural extension** (new operation on the carrier) and **action extension** (new operation involving the carrier without changing its operational set). Confirmations 1–4 are all structural extensions, each introducing a new predicate on the carrier and so requiring a new apparatus instance. Confirmation 5 is an action extension, which does not introduce a new carrier-level predicate and so does not require a new apparatus instance.

The pattern generalises beyond v1.0.0: any future typeclass that adds an action without introducing a new carrier-level predicate would similarly reuse an existing apparatus instance. Examples in §II.6 include operational ring homomorphisms, operational subrings, operational algebras. The framework, by Confirmation 5, accommodates the distinction structurally.

III.4. Methodological implication

Finding A3, across five confirmations, supports the position that the apparatus framework of VR-Apparatus is **structurally general across algebraic structures**, not analysis-specific. VR-Audit demonstrated the apparatus on analysis (Hahn-Banach as Mode B audit); VR-Algebra demonstrates the apparatus on algebra (groups, rings, fields, modules). Two domain demonstrations is not proof of universality — generality across all of mathematics — but it is substantive support for the claim that the framework is applicable beyond its original domain of formulation.

The methodological implication for downstream work: extending the operational tracking discipline to additional algebraic structures (polynomial rings, matrix rings, algebras over fields, modules over non-commutative rings, etc.) follows the same architectural pattern. The apparatus instance is either declared (for structural extension) or reused (for action extension); the closure axioms are stated for the new operations; the typeclass is constructed.

What is **not** asserted by Finding A3:

- That the apparatus framework is the only possible framework for operational tracking. It is one framework; alternatives may exist.
- That every conceivable mathematical structure admits a corresponding operational typeclass. The framework is shown to work in algebra (this work) and analysis (VR-Audit); other domains await demonstration.
- That the apparatus instance is unique up to isomorphism. The instance depends on the choice of predicate; different predicates give different instances on the same carrier.

What is asserted:

- The framework, as formulated, applies to algebraic structures without modification.
- The framework instance can be reused across typeclasses sharing a carrier with the same operational predicate (Confirmation 5).
- The methodological pattern of structural vs action extension is observed and exploited in the apparatus instance design.

Part IV develops the complete enumeration of Mode A theorems entailed by the closure axioms of Part II. The apparatus instances established here provide the typeclass infrastructure within which the Mode A theorems are stated and proved.

Part IV. Mode A Theorems

IV.0. Introduction

Part II constructed the operational typeclasses with their closure axioms. Part III established that the apparatus framework registers on each typeclass without modification. Part IV enumerates the Mode A theorems — closure of algebraic operations under operationality, expressed in the apparatus's standard form.

Mode A, in VR-Apparatus, is the pattern: an operation $f : T \rightarrow T$ (unary) or $f : T \rightarrow T \rightarrow T$ (binary, homogeneous) preserves a predicate $P : T \rightarrow \text{Prop}$ on operational inputs. Formally: $\text{IsModeAOp } f$ asserts $\forall x, P\ x \rightarrow P\ (f\ x)$; $\text{IsModeAOp}_2\ f$ asserts $\forall x\ y, P\ x \rightarrow P\ y \rightarrow P\ (f\ x\ y)$. The Mode A statement is the apparatus-level theorem; the closure axiom of the operational typeclass is the corresponding low-level statement. In each case, the Mode A theorem is a one-line invocation of the closure axiom.

This part enumerates the Mode A theorems for each operational typeclass. Subsections IV.1–IV.5 follow the order of the typeclasses; IV.6 develops the heterogeneous module case (Finding A19); IV.7 summarises.

IV.1. Additive operations

The Mode A theorems for `OperationalAddGroup G` follow the closure axioms directly. The basic operations:

- `add_isModeAOp : IsModeAOp₂ (· + · : G → G → G)` — addition is Mode A binary. The proof is `fun a b ha hb => add_isOperational a b ha hb`.
- `neg_isModeAOp : IsModeAOp (- · : G → G)` — negation is Mode A unary. The proof is `fun a ha => neg_isOperational a ha`.
- `sub_isModeAOp : IsModeAOp₂ (· - · : G → G → G)` — subtraction is Mode A binary. The proof is `fun a b ha hb => sub_isOperational a b ha hb`.

Subtraction is treated as a primitive operation rather than derived from addition and negation, consistent with the closure axiom design (see §II.1).

The scalar action by natural and integer numbers is a different structural case. The scalar argument has type $\mathbb{N}$ or $\mathbb{Z}$, not G ; the resulting map is $G \rightarrow G$ for a fixed scalar. The theorems are:

- `nsmul_isOperational : ∀ a : G, IsOperational a → ∀ n : ℕ, IsOperational (n • a)` — natural-number scalar multiplication preserves operability, by induction on n . The base case $0 • a = 0$ invokes `zero_isOperational`; the inductive case $(n+1) • a = a + n • a$ invokes `add_isOperational` and the induction hypothesis.
- `zsmul_isOperational : ∀ a : G, IsOperational a → ∀ n : ℤ, IsOperational (n • a)` — integer scalar multiplication preserves operability, by case analysis on the sign of n . The non-negative case reduces to `nsmul_isOperational`; the negative case `Int.negSucc n • a = -((n+1) • a)` invokes `neg_isOperational` and `nsmul_isOperational`.

These theorems use the unary form `IsModeAOp` applied to the family of maps parameterised by the scalar. The full heterogeneous binary form arises for module scalar multiplication (§IV.6).

The `zsmul` proof has a small engineering quirk relative to its multiplicative counterpart `zpow`: while `zpow` admits a bare `exact` in the case decomposition, `zsmul` requires an explicit rewrite step through `mathlib`’s `natCast_zsmul` and `negSucc_zsmul` lemmas. The mathematical content is the same; the proof mechanics differ. This is documented as Finding A18.

IV.2. Ring operations

The Mode A theorems for `OperationalRing R` add to the additive theorems (inherited through the bridge) the multiplicative content:

- `mul_isModeAOp : IsModeAOp₂ (· * · : R → R → R)` — multiplication is Mode A binary. The proof is `fun a b ha hb => mul_isOperational a b ha hb`.
- `npow_isOperational : ∀ a : R, IsOperational a → ∀ n : ℕ, IsOperational (a^n)` — natural-number power preserves operability, by induction on n . The base case $a^0 = 1$ invokes `one_isOperational`; the inductive case $a^{(n+1)} = a * a^n$ invokes `mul_isOperational` and the induction hypothesis.

The multiplicative identity $1 : R$ is not subject to a Mode A theorem of the form `f(x)` preserves operability; it is a constant whose operational status is asserted by the closure axiom `one_isOperational`.

IV.3. Multiplicative group operations

The Mode A theorems for `OperationalGroup G` (multiplicative) mirror the additive theorems of §IV.1 structurally, with $+$ $\rightarrow$ $*$, $-$ $\rightarrow$ $^{-1}$, etc.:

- `mul_isModeAOp` — multiplication is Mode A binary.
- `inv_isModeAOp` : `IsModeAOp ( $\cdot^{-1}$  :  $G \rightarrow G$ )` — inverse is Mode A unary.
- `div_isModeAOp` : `IsModeAOp2 ( $\cdot / \cdot$  :  $G \rightarrow G \rightarrow G$ )` — division is Mode A binary.

For multiplicative powers:

- `npow_isOperational` — natural-number multiplicative power preserves operability.
- `zpow_isOperational` : $\forall a : G, \text{IsOperational } a \rightarrow \forall n : \mathbb{Z}, \text{IsOperational } (a^n)$ — integer multiplicative power preserves operability, by case analysis on the sign of n . As noted in §IV.1, this proof admits a slightly simpler form than the additive `zsmul` counterpart.

IV.4. Field operations

The Mode A theorem specific to `OperationalField K`:

- `inv_isModeAOp_field` : `IsModeAOp ( $\cdot^{-1}$  :  $K \rightarrow K$ )` — inverse on a field is Mode A unary.

The mathematical content is one application of the field’s `inv_isOperational` closure axiom. The Lean axiom audit, however, registers [`propext`, `Classical.choice`, `Quot.sound`] — the full ceiling, with `Classical.choice` propagated from `mathlib`’s encoding of `Inv K` resolution through the apparatus import chain. This is Finding A15/A16 and is discussed in Part VII; the mathematical content of the theorem itself is constructive.

The other operations on a field — addition, multiplication, etc. — inherit their Mode A theorems from `OperationalRing K` and `OperationalAddGroup K` through the bridge instances. No new Mode A theorems are declared at the field level for these operations.

IV.5. Module scalar action — heterogeneous case

The Mode A theorem for the scalar action of an operational module differs structurally from the homogeneous cases of §§IV.1–IV.4. The scalar action $\cdot : R \rightarrow M \rightarrow M$ has input types R (scalar) and M (module element) that are distinct, with output in M . The Mode A pattern `IsModeAOp2 f` for $f : T \rightarrow T \rightarrow T$ does not apply: the function is heterogeneous, not homogeneous.

The applicable Mode A pattern, given a fixed operational scalar $r : R$, is the unary form on M :

- `smul_isModeAOp` — for `[OperationalModule R M]` and a fixed operational $r : R$, the right-scalar map $r \cdot \cdot : M \rightarrow M$ is Mode A unary on M with respect to the additive group predicate.

Formally:

```
theorem smul_isModeAOp {R M : Type*}
  [Ring R] [AddCommGroup M] [Module R M]
  [OperationalRing R] [OperationalAddGroup M] [OperationalModule R M]
  {r : R} (hr : OperationalRing.IsOperational r) :
  IsModeAOp (P := OperationalAddGroup.IsOperational (G := M)) (r · ·) :=
  fun _ hm => OperationalModule.smul_isOperational hr hm
```

For an operational scalar r , the map $r \cdot \cdot : M \rightarrow M$ is Mode A unary. The Mode A certificate is the typeclass field `smul_isOperational` applied to the scalar’s operational hypothesis and the unary input’s operational hypothesis. The proof is one line.

This is the **first heterogeneous binary operation** in the VR cycle to receive a Mode A treatment. The structural form differs from the homogeneous cases of §§IV.1–IV.4 — specifically, the Mode A statement is unary on M for a fixed r , rather than binary on $M \rightarrow M \rightarrow M$. The change is structural: the underlying mathematical content has two distinct types, and the Mode A pattern is applied with the right specialisation.

The methodological observation: the apparatus’s Mode A definition admits both homogeneous and heterogeneous binary operations, with the heterogeneous case parameterised by the “other” argument. This generalisation was not anticipated in the initial planning of the cycle; the Mode A treatment of the module scalar action was originally specified in the form `IsModeAOp2 ( $\cdot \cdot \cdot : R \rightarrow M \rightarrow M$ )`, which is type-incorrect. The error was identified during implementation and corrected to the unary form for a fixed scalar. The episode — recognition discipline applied to the plan itself, not to code — is recorded as Finding A19 and discussed further in Part VI.

IV.6. Mode A theorem catalog

The complete catalog of Mode A theorems across the operational typeclasses of v1.0.0:

Theorem	Typeclass	Operation	Form	Axiom profile
<code>add_isModeAOp</code>	<code>AddGroup</code>	binary	binary homogeneous	[]
<code>neg_isModeAOp</code>	<code>AddGroup</code>	unary	unary	[]
<code>sub_isModeAOp</code>	<code>AddGroup</code>	binary	binary homogeneous	[]
<code>nsmul_isOperational</code>	<code>AddGroup</code>	parameterised $\mathbb{N}$ -family	unary per scalar	[]
<code>zsmul_isOperational</code>	<code>AddGroup</code>	parameterised $\mathbb{Z}$ -family	unary per scalar	[]
<code>mul_isModeAOp</code> (ring)	<code>Ring</code>	binary	binary homogeneous	[]
<code>npow_isOperational</code> (ring)	<code>Ring</code>	parameterised $\mathbb{N}$ -family	unary per scalar	[]
<code>mul_isModeAOp</code> (mul-group)	<code>MulGroup</code>	binary	binary homogeneous	[]
<code>inv_isModeAOp</code> (mul-group)	<code>MulGroup</code>	unary	unary	[]
<code>div_isModeAOp</code>	<code>MulGroup</code>	binary	binary homogeneous	[]
<code>npow_isOperational</code> (mul-group)	<code>MulGroup</code>	parameterised $\mathbb{N}$ -family	unary per scalar	[]
<code>zpow_isOperational</code>	<code>MulGroup</code>	parameterised $\mathbb{Z}$ -family	unary per scalar	[]
<code>inv_isModeAOp_field</code>	<code>Field</code>	unary	unary	[propext, Classical.choice, Quot.sound]

Theorem	Typeclass	Operation	Form	Axiom profile
<code>smul_isModeAOp</code>	Module	parameterised R-family	unary per scalar	[]

Fourteen Mode A theorems. Thirteen at axiom profile [] (axiom-free pure algebraic content); one at the full ceiling [`propext`, `Classical.choice`, `Quot.sound`] for the field inverse, with the `Classical.choice` dependency structural to `mathlib`’s encoding (Part VII).

The cataloguing is exhaustive at the v1.0.0 algebraic level: every standard operation of the algebraic hierarchy is provided with a Mode A theorem. The Mode A theorems support the substantive Mode B audit of Part V and the methodological observations of Parts VI–VIII.

IV.7. Summary of Part IV

Fourteen Mode A theorems are stated and proved, exhaustively covering the standard operations of the algebraic hierarchy: addition, negation, subtraction, multiplication, inverse, division, integer and natural-number powers (additive and multiplicative), and scalar multiplication for modules. All theorems have one-line proofs from the closure axioms of their respective operational typeclasses; the proofs are immediate over the apparatus once the closure axiom is stated.

Two structural observations distinguish this enumeration from a routine listing:

- (1) The heterogeneous case of module scalar action (§IV.5) is the first heterogeneous binary operation in the VR cycle to receive a Mode A treatment. The Mode A pattern accommodates the case via specialisation to the unary form for a fixed scalar. This is recorded as Finding A19.
- (2) The field-inverse Mode A theorem (§IV.4) is the only Mode A theorem of the cycle to carry a non-trivial axiom profile (`Classical.choice` through `mathlib`’s encoding of `Inv K`). The phenomenon is structural and is discussed in Part VII as Finding A15/A16.

The next part — Part V — develops the substantive Mode B audit conducted in the cycle: the image of an operational subgroup under an operational homomorphism is an operational subgroup. The Mode B audit complements the Mode A enumeration of the present part by demonstrating a substantive mathematical theorem proved over the apparatus, rather than a closure property of an operation.

Part V. Mode B Audit

V.0. Introduction

Part IV enumerated the Mode A closure theorems — closure of operations under operationality. Part V develops the substantive Mode B audit of the cycle: a theorem of algebraic content proved over the apparatus, rather than a closure property of an operation.

Mode B in VR-Apparatus is the pattern: a theorem stating that an operation, applied to operational inputs, produces an operational output that participates in a substantive mathematical relation. Where Mode A asserts “operational input $\rightarrow$ operational output”, Mode B asserts “operational input $\rightarrow$ operational output that satisfies a stated property”. The Mode B theorem combines closure with substantive content.

The substantive Mode B audit of v0.2.0 is:

If $\varphi : G \rightarrow H$ is an operational additive group homomorphism and S is an operational subgroup of G , then the image $\varphi(S)$ is an operational subgroup of H .

This is `image_isOperationalAddSubgroup_isModeB0p` in the Lean formalisation. The theorem states substantive mathematical content (image of subgroup is subgroup) combined with operational tracking (image of operational subgroup under operational homomorphism is operational subgroup). Part V develops the theorem, its proof, and its contrast with the Mode B audit conducted in VR-Audit (Hahn-Banach).

Contents: V.1 states the theorem precisely; V.2 develops the proof; V.3 discusses the role of `AddSubgroup.mem_map` as constructive witness; V.4 contrasts with VR-Audit’s Hahn-Banach treatment (Finding A11); V.5 summarises.

V.1. The theorem

Let G and H be additive groups with operational typeclass instances:

```
variable {G H : Type*} [AddGroup G] [AddGroup H]
variable [OperationalAddGroup G] [OperationalAddGroup H]
```

An **operational additive group homomorphism** is an `AddGroup` homomorphism $\varphi : G \rightarrow H$ such that for every operational $a : G$, the image φa is operational in H . This is captured by the auxiliary hypothesis:

```
variable (hW : ∀ a : G, OperationalAddGroup.IsOperational a →
          OperationalAddGroup.IsOperational (φ a))
```

The hypothesis hW is the witness data of the homomorphism’s operability. Given such a homomorphism, the substantive Mode B claim:

Theorem (image of operational subgroup). Let $S : \text{AddSubgroup } G$ satisfy `IsOperationalSubgroup S`. Then $S.\text{map } \varphi$, the image of S in H , satisfies `IsOperationalSubgroup (S.map φ)`.

Here `IsOperationalSubgroup S` is the predicate that every element of S (as an element of G) is operational. Equivalently: the operational predicate restricted to the subgroup carrier holds for every element.

The theorem combines two mathematical contents:

1. **Subgroup content:** the image of a subgroup under a group homomorphism is a subgroup. This is a standard result of group theory, supplied by `mathlib`’s `AddSubgroup.map`.
2. **Operational content:** the image of an operational subgroup under an operational homomorphism is an operational subgroup. This is the new content introduced by operational tracking.

The Mode B statement is the conjunction of the two contents — the theorem asserts that the image is both a subgroup (provided by `mathlib`) and operational (provided by the apparatus).

V.2. The proof

The proof structure:

```
intro S hS hW y hy
rw [AddSubgroup.mem_map] at hy
obtain ⟨x, hxS, rfl⟩ := hy
exact hW x (hS x hxS)
```

Reading from top:

- `intro S hS hW y hy` — introduce the subgroup S , the hypothesis $hS : \text{IsOperationalSubgroup } S$, the homomorphism operationality hW , an arbitrary element y of the image, and the hypothesis hy that y is in the image.
- `rw [AddSubgroup.mem_map] at hy` — rewrite hy using `mathlib`’s characterisation of subgroup-image membership: $y \in S.\text{map } \varphi \leftrightarrow \exists x \in S, \varphi x = y$.
- `obtain (x, hxS, rfl) := hy` — destruct the existential to obtain the preimage x , its membership in S (hxS), and the equation $\varphi x = y$ (resolved by `rfl` after substitution).
- `exact hW x (hS x hxS)` — apply the homomorphism’s operationality to x , using the subgroup’s operationality to provide the operational hypothesis on x .

The substantive content of the proof is the last line: $hW x (hS x hxS)$. It states: the homomorphism takes x (which is operational by $hS x hxS$) to an operational element (by hW). The element to which x is taken is precisely y (by the destructured equation). Therefore y is operational.

The proof is **one inference step over the apparatus** plus the destructuring of `mathlib`’s `mem_map` characterisation. The apparatus contribution — what the operational tracking discipline adds to the proof — is the $hW x (hS x hxS)$ line. Everything else is bookkeeping over `mathlib`’s existing infrastructure.

V.3. The constructive witness

The decisive structural feature of the proof is the **constructive witness** provided by `AddSubgroup.mem_map`. The characterisation $y \in S.\text{map } \varphi \leftrightarrow \exists x \in S, \varphi x = y$ gives, for each y in the image, an explicit preimage x . The existential is not “existence somewhere”; it is “existence with a witness extractable by `obtain`”.

Once the witness is obtained, the apparatus content reduces to: “the homomorphism is operational, so it preserves operationality from x to y ”. This is a single application of the homomorphism operationality hypothesis.

The constructive witness is supplied by **mathlib**, not by the operational tracking. The operational tracking exploits the witness — it does not construct it. This is the structural feature of the audit: the apparatus tracks operationality, and where `mathlib` provides constructive content, the apparatus operates constructively as well.

The axiom profile of the audit theorem is `[propext]` — propositional extensionality, but no `Classical.choice` and no `Quot.sound`. The pure algebraic content is constructive throughout. The apparatus contributes the operational tracking; the constructive content is inherited from `mathlib`’s subgroup theory.

V.4. Contrast with VR-Audit (Hahn-Banach)

VR-Audit conducted a substantive Mode B audit in analysis: the Hahn-Banach extension theorem treated as Mode B. The theorem there:

If $\varphi : V \rightarrow \mathbb{R}$ is a bounded linear functional on a subspace V of a normed space X , then φ extends to a bounded linear functional on all of X with the same norm. If the data is operational (computable subspace, computable functional, computable norm bound), then the extension is operational.

The mathematical content of Hahn-Banach is one of the central existence theorems of functional analysis; the operational content is to extract a computable extension when the data is computable. The two contents combine into a Mode B audit.

The structural contrast with the present audit is in three dimensions:

Witness construction.

The image-of-subgroup audit obtains its witness from `mathlib's AddSubgroup.mem_map`, which provides the preimage directly via the existential. The witness is constructive, obtained by destructuring.

The Hahn-Banach audit obtains its “witness” from Zorn’s lemma (or equivalent — Hahn-Banach is equivalent to the Boolean Prime Ideal Theorem, strictly weaker than the axiom of choice but still non-constructive). The witness is the extension functional, selected from a non-empty set of admissible extensions; no explicit construction is provided.

Axiom profile.

The image-of-subgroup audit has axiom profile `[propext]` — no `Classical.choice`, no non-constructive content. The proof is constructive throughout.

The Hahn-Banach audit has axiom profile `[propext, Classical.choice, Quot.sound]` — the full ceiling. The `Classical.choice` enters with Zorn’s lemma; the audit accepts it and tracks the operational content where it survives.

Proof structure.

The image-of-subgroup audit’s substantive Mode B step is one line: `exact hW x (hS x hXS)`. The audit consists of bookkeeping (destructuring the existential, applying the hypothesis) plus the single apparatus invocation.

The Hahn-Banach audit’s substantive Mode B step involves multiple supporting lemmas (existence of maximal extensions, norm preservation under extension, dominated convergence-style arguments). The audit is several lemmas long, with the apparatus tracking operational content through the classical machinery.

The structural contrast is recorded as `Finding A11`. The interpretation: algebraic Mode B audits, when the mathematical content has a constructive witness (such as `mem_map`), reduce to one apparatus invocation. Analytic Mode B audits, when the mathematical content requires classical existence selection (such as Hahn-Banach), require several supporting lemmas and accept `Classical.choice` in the audit’s axiom profile.

This is not a defect of either approach. It reflects the structural mathematics:

- Subgroup theory is constructive (the image of an element is computable from the element; the image of a subgroup is the set of images).
- Hahn-Banach is non-constructive (the extension is not, in general, computable from the data; classical existence selection is necessary).

The apparatus tracks operability through both kinds of mathematics. In constructive content, the apparatus operates constructively. In classical content, the apparatus accepts classical content and tracks where operability survives.

The methodological observation: the apparatus is **operationally honest** in the sense that it does not hide where `Classical.choice` is invoked. The axiom audit of every theorem records its dependency on Classical machinery. When a theorem is constructive (image-of-subgroup), the axiom audit reflects this; when a theorem is classical (Hahn-Banach), the audit reflects this also. The apparatus does not collapse the distinction; it tracks it.

V.5. Summary of Part V

One substantive Mode B audit is conducted: the image of an operational subgroup under an operational homomorphism is an operational subgroup. The theorem is `image_isOperationalAddSubgroup_isModeB0p` at axiom profile `[propext]`.

The proof is one inference step over the apparatus, after `AddSubgroup.mem_map` supplies the existential witness constructively. The constructive content is inherited from `mathlib`'s subgroup theory; the apparatus contributes the operational tracking.

The structural contrast with VR-Audit's Hahn-Banach Mode B audit — algebraic Mode B reduces to one apparatus invocation with constructive witness; analytic Mode B requires multiple lemmas and accepts `Classical.choice` — is recorded as Finding A11.

The methodological observation: the apparatus is **operationally honest**; it tracks `Classical.choice` dependencies where they exist and operates constructively where they do not. The same apparatus serves both constructive (algebraic) and classical (analytic) mathematics, with the axiom profile of each theorem reflecting its true dependencies.

Part VI develops the recognition discipline narrative across the cycle: the seven applications of the bidirectional pattern (five removals, one introduction, one plan correction) that constitute the methodological signature of the work.

Part VI. Recognition Discipline

VI.0. Introduction

The construction of Part II — five operational typeclasses with closure axioms, bridges, and apparatus instances — was developed across four numbered cycles (v0.1.0 through v0.4.0) plus a polish cycle (v1.0.0). At every stage, the construction was subject to a methodological discipline: abstractions are introduced when concrete use cases justify them, and abstractions are removed when no concrete use case has emerged for them. This is the **recognition discipline**, anticipated in VR-Apparatus and applied bidirectionally in the present cycle.

The pattern is observed seven times in v1.0.0:

- **Five removals** of preemptive abstractions when no instances yet justified them.
- **One introduction** of a previously removed abstraction when an instance subsequently justified it (closing the loop).
- **One plan-level correction** when the architectural form of a planned theorem was found to be incorrect during implementation, refined to the correct form before code was written.

The seven applications together constitute the methodological signature of the cycle. They are catalogued individually in the findings catalog (Findings F11, A0, A6, A9, two Stage 5 v0.2.0 omissions, A12, A19) and discussed thematically in this part.

Contents: VI.1 states the discipline; VI.2 narrates the five removal applications; VI.3 develops the introduction application (closing the loop, Finding A12); VI.4 discusses the plan correction application (Finding A19); VI.5 develops the methodological implication.

VI.1. The discipline

The recognition discipline states:

An abstraction (typeclass, structure, lemma, definition) is introduced into the formalisation when, and only when, a concrete use case justifies it. An abstraction is removed when no concrete use case has emerged for it within the scope of the cycle.

The discipline is symmetric in two directions:

Removal direction: when developing a new typeclass, the implementer may be tempted to include abstractions “that will likely be needed later”. The discipline counsels against this: abstractions without instances are not abstractions of anything. They are placeholder syntax that will need refactoring once concrete use cases emerge. Better to omit them and revisit when needed.

Introduction direction: when a concrete use case for a previously removed abstraction emerges, the abstraction is introduced — without prejudice from the prior removal. The prior removal was correct at the time (no instance justified it then); the present introduction is correct now (the instance has emerged).

The discipline distinguishes abstraction from speculation. Abstraction reflects observed structure: similar patterns across multiple concrete cases admit a unified treatment. Speculation reflects expected structure: similar patterns might emerge later, so the abstraction is provided in advance. The discipline accepts abstraction and rejects speculation.

VI.2. The five removals

F11 — VR-Apparatus context (carry-over from earlier work).

The VR-Apparatus cycle introduced and subsequently removed a generic `Register` typeclass — an abstraction over multiple-implementation registration. At the time of removal, no use case had emerged that benefited from the generic version. The specific registration patterns sufficed. F11 was the inaugural removal application in the VR cycle and provided the precedent for subsequent removals.

A0 — `OperationalGroup` (multiplicative).

The initial v0.1.0 plan included `OperationalGroup` — a multiplicative analogue of `OperationalAddGroup`. At v0.1.0, no concrete instance of an operational multiplicative group had been identified: the integers and $\mathbb{Z}/n$ are additive (not multiplicative) groups in the operational context; rings and fields use multiplication but are not “multiplicative groups” in the technical sense (`Group` typeclass). The typeclass was omitted, with the expectation that it might be reintroduced when an instance emerged.

A6 — Bundled `OperationalAddSubgroup` structure.

The subgroup predicate `IsOperationalSubgroup S` was originally accompanied by a draft of a bundled structure `OperationalAddSubgroup G` — a record carrying both the subgroup data and its operational predicate. By v0.1.0 it was clear that no use case required the bundle: the predicate suffices for stating theorems, and the bundle adds no information not derivable from the predicate. The bundled structure was omitted; the predicate remained.

A9 — `OperationalCommRing`.

When `OperationalRing` was introduced in v0.2.0, a natural extension is `OperationalCommRing` — for commutative rings. The recognition discipline applied: commutativity adds no operational content (the closure

axiom for multiplication is the same axiom for commutative and non-commutative rings). No use case had emerged where commutativity at the operational level was needed. The typeclass was omitted.

Two Stage 5 v0.2.0 omissions.

Stage 5 of v0.2.0 considered two bridge theorems that would have provided redundant low-level lemmas (`one_isOperational_bridge` — bridging ring’s one to multiplicative group’s one; `mul_chain_isOperational` — composing several multiplicative closure axioms in sequence). Each was found to be derivable from existing axioms by standard Lean tactics, providing no new content. The theorems were omitted from the stage; downstream code constructs the relevant chains directly when needed.

These five removals share a common pattern: an abstraction was considered, the use case was sought, the use case was not found within scope, the abstraction was omitted. The discipline thus reflects a willingness to leave abstractions on the table until they justify their cost.

VI.3. The introduction — closing the loop

A12 — `OperationalGroup` revived.

V0.3.0 introduced `OperationalField` (§II.4). The field-of-units construction provided a natural instance of an operational multiplicative group: the units $K^\times$ of an operational field K form a multiplicative group, and the operability of $K^\times$ is inherited from the operability of K . This is the concrete instance that v0.1.0 lacked.

At this point the recognition discipline applies in reverse: the abstraction omitted in v0.1.0 (Finding A0) is now justified by an instance, and so it is introduced. `OperationalGroup` is added to v0.3.0; its first instance is the bridge `toOperationalGroupUnits` from `OperationalField`.

The introduction closes the loop opened by A0. The methodological observation: the discipline is not “omit abstractions on principle”. It is “omit abstractions until justified, then introduce them when justified”. The Tax of revisiting is small (a new typeclass declaration plus its instance); the benefit is principled: abstractions in the final form of the cycle each correspond to at least one concrete instance.

The loop closure $A0 \rightarrow A12$ is the **methodological signature** of the cycle. Five removals plus one introduction document a complete bidirectional cycle: abstractions removed when not justified, abstractions introduced when justified, the same abstraction passing through both phases as the use case emerged.

VI.4. The plan correction

A19 — Heterogeneous binary operation.

The Stage 5 plan of v0.4.0 specified the Mode A theorem for module scalar multiplication as `IsModeA0p2` ($\cdot : R \rightarrow M \rightarrow M$). The implementer (in the parent-child review architecture) identified that the form is type-incorrect: `IsModeA0p2` `f` requires `f : T → T → T` with a single predicate `P : T → Prop`, but $\cdot : R \rightarrow M \rightarrow M$ is heterogeneous, with input types R and M distinct. The architectural form was refined to `IsModeA0p` (`r · : M → M`) for fixed operational `r : R` — unary for a fixed scalar, parameterised by `r`.

The discipline applies here to the plan itself, not to code. The original plan specified an architecture; the implementation phase identified the architecture as incorrect; the architecture was corrected before any code was written. The seventh application of the recognition discipline.

The methodological observation: the parent-child review architecture (architectural reviewer + implementer) admits this correction structurally. The reviewer’s plan is provisional until the implementer verifies it against

the actual mathematical content. When the verification fails, the plan is refined. The implementer is not bound to follow incorrect plans; the reviewer is not insulated from correction. The architecture is collaborative, with both parties contributing to the correctness of the final form.

Finding A19 thus extends the recognition discipline beyond code-level abstractions to plan-level abstractions. The principle is the same: an architectural decision is correct if a concrete mathematical use case is consistent with it; if the use case is not consistent with the planned architecture, the architecture is incorrect and is revised. The plan-correction application is the natural extension of the discipline to the level of planning.

VI.5. Methodological implication

Seven applications of the recognition discipline document a methodological signature for the cycle:

- The construction process is not a one-pass write-down of a complete plan.
- The construction is iterative: abstractions are introduced and removed as use cases are observed.
- The final form of the construction (v1.0.0) reflects the use cases that emerged; not the abstractions that were initially conjectured.
- The discipline applies bidirectionally (removal and introduction) and across levels (code and plans).

The methodological position underlying the discipline is operational in character. An abstraction is, structurally, a generalisation of patterns observed in multiple instances. Without instances, the “abstraction” abstracts from nothing — it is a syntactic structure without mathematical content. The discipline reflects the position: mathematics is in the instances; abstraction follows.

This is a position in formal mathematics methodology, parallel in spirit to the operational tracking discipline applied to mathematical content. Just as the apparatus tracks which elements have operational construction witnesses, the recognition discipline tracks which abstractions have instance witnesses. Both are forms of careful tracking; both reject speculation in favour of demonstrated structure.

The methodological signature established in this cycle — bidirectional recognition discipline applied seven times — supports the position that the apparatus framework can be developed iteratively and honestly. Each abstraction in the final v1.0.0 has at least one concrete instance; each instance was the trigger for the abstraction’s introduction; each abstraction not in v1.0.0 was omitted because no instance emerged for it within scope.

The discipline can be applied to extensions of the cycle in the same form. Polynomial rings, matrix rings, and other potential additions (§IX) will be added when concrete instances emerge that justify them; until then, they remain potential. The methodological principle is preserved across extensions.

VI.6. Summary of Part VI

Seven applications of the recognition discipline are documented:

- Five removals (F11 from VR-Apparatus context, A0, A6, A9, two Stage 5 v0.2.0 omissions) — abstractions omitted when no instances justified them.
- One introduction (A12) — `OperationalGroup` revived when field-of-units provided an instance, closing the loop opened by A0.
- One plan-level correction (A19) — heterogeneous module scalar multiplication refined from `IsModuleA0p2` to `IsModuleA0p` for fixed scalar, before code was written.

The applications collectively constitute the methodological signature of the cycle: bidirectional recognition discipline observed across code and plans, seven times applied, principle preserved throughout.

The methodological observation: abstractions are tracked by their instance justifications; absence of instances is grounds for removal; emergence of instances is grounds for introduction; the cycle’s final form (v1.0.0) reflects the observed mathematical content, not speculative architectural commitment.

Part VII develops the axiom analysis of the cycle: the three-tier hierarchy, Finding A10 (ceiling = carrier property), and Finding A15/A16 (structural `Classical.choice` in field-inverse reasoning).

Part VII. Axiom Analysis

VII.0. Introduction

The operational tracking discipline imposes no axioms on the underlying mathematics; it imposes a tracking discipline on which operations preserve operational status. In Lean 4 (the implementation language of the present formalisation), every theorem can be inspected with `#print axioms` to determine its dependency on the meta-theoretic axioms — propositional extensionality (`propext`), `Classical.choice`, and quotient soundness (`Quot.sound`). These dependencies, when examined systematically across the cycle, exhibit structural patterns of independent interest.

Part VII documents the patterns. VII.1 presents the three-tier axiom hierarchy. VII.2 develops Finding A10: the axiom ceiling of every algebraic structure on a given carrier is determined by the carrier type, not the algebraic depth. VII.3 develops Finding A15: two objects in the cycle (`inv_isModeAOp_field` and `OperationalField.toOperationalGroupUnits`) carry `Classical.choice` for structural reasons traceable to `mathlib`’s encoding of `Inv K` resolution. VII.4 documents the propagation of A15 to $\mathbb{Q}$ -Module objects via the bridge chain. VII.5 frames the honest constructivity statement of the cycle.

VII.1. The three-tier axiom hierarchy

Across the 64 public objects of the v1.0.0 cycle, three axiom profile tiers appear:

Tier 1 — [] (axiom-free): theorems and definitions that depend on no meta-theoretic axiom. Pure algebraic content, derivable directly from definitions and lemmas without invoking quotient soundness, propositional extensionality, or classical choice.

Tier 2 — [`propext`] and [`propext`, `Quot.sound`]: theorems that depend on propositional extensionality and (in many cases) on quotient soundness. The dependencies enter through the carrier types: $\mathbb{Z}$ involves `propext` through subtype constructions; $\mathbb{Z} \bmod n$ involves `Quot.sound` through the quotient definition $\mathbb{Z} / n\mathbb{Z}$; $\mathbb{Q}$ involves both through similar constructions.

Tier 3 — [`propext`, `Classical.choice`, `Quot.sound`]: theorems that depend on the full ceiling including `Classical.choice`. Two distinct sources contribute: (i) the carrier-type ceiling for $\mathbb{Q}$ -based objects (`mathlib`’s encoding of `Rat.instField` propagates `Classical.choice` through the field instance), and (ii) the structural `Classical.choice` in field-inverse reasoning documented as Finding A15.

The distribution across the 64 objects:

Tier	Count	Examples
[]	~25	Mode A theorems (generic), typeclass definitions, basic constructions
[propext]	~15	$\mathbb{Z}$ instances and $\mathbb{Z}$ -based theorems
[propext, Quot.sound]	~15	ZMod n instances and ZMod-based theorems
[propext, Classical.choice, Quot.sound]	~9	$\mathbb{Q}$ instances and $\mathbb{Q}$ -based theorems, two field-inverse objects (A15)

The tier structure is hierarchical: every theorem at a higher tier subsumes the lower-tier dependencies. The audit reflects the actual dependencies, not a global ceiling.

VII.2. Finding A10 — ceiling = carrier property

Finding A10 states: the axiom ceiling of every algebraic structure built on a given carrier type is identical to the carrier type’s intrinsic ceiling. The ceiling is determined by the carrier, not by the algebraic depth of the structure built on it.

The finding is confirmed structurally across four algebraic levels (group, ring, field, module) on two carrier types ($\mathbb{Z}$, $\mathbb{Q}$):

Carrier	AddGroup	Ring	Field	Module
$\mathbb{Z}$	[propext]	[propext]	—	[propext]
$\mathbb{Q}$	[propext, C, Q]	[propext, C, Q]	[propext, C, Q]	[propext, C, Q]

(Where [propext, C, Q] abbreviates [propext, Classical.choice, Quot.sound].)

The pattern is clear: every algebraic structure on $\mathbb{Z}$ has axiom ceiling [propext], regardless of whether it is an additive group, a ring, or a module. Every algebraic structure on $\mathbb{Q}$ has axiom ceiling [propext, Classical.choice, Quot.sound]. The algebraic depth (group $\rightarrow$ ring $\rightarrow$ field $\rightarrow$ module) does not escalate the ceiling.

The structural reason: the algebraic typeclasses extend the carrier type’s existing structure. The carrier’s axiom dependencies are inherited by every typeclass built on it. The closure axioms added by the operational typeclasses are typically axiom-free (the closure is provable by direct unfolding of the predicate, which is True in current instances). Therefore the algebraic typeclass instance inherits exactly the carrier’s ceiling, not more, not less.

The methodological implication: when extending the cycle to new carrier types (real numbers, complex numbers, p-adics, etc.), the axiom ceiling of the resulting operational structure is determined by mathlib’s encoding of the new carrier, not by the cycle’s tracking discipline itself. The discipline adds no axiom dependency beyond what the carrier already requires.

$\mathbb{Z} \text{Mod } n$ is an intermediate case. Each $\mathbb{Z} \text{Mod } n$ carrier has axiom ceiling `[propext, Quot.sound]` — the quotient soundness enters with the quotient construction $\mathbb{Z} / n\mathbb{Z}$; propositional extensionality enters through general subtype machinery. Finding A10 holds: every algebraic structure on $\mathbb{Z} \text{Mod } n$ has the same ceiling.

The exception to Finding A10 — and the only exception — is Finding A15, two specific theorems involving field inverse that pull `Classical.choice` for structural reasons distinct from the carrier-type ceiling. These are discussed in §VII.3.

VII.3. Finding A15 — structural `Classical.choice` in field-inverse reasoning

Finding A15 states: two specific theorems in the cycle (`inv_isModeAOp_field` and `OperationalField.toOperationalGroupUnits`) have axiom profile `[propext, Classical.choice, Quot.sound]` for structural reasons, not because of constructive failures in the underlying mathematics.

The finding emerged in v0.3.0 when these two theorems were observed to carry `Classical.choice` despite having one-line constructive proofs:

- `inv_isModeAOp_field` — Mode A theorem stating that field inverse preserves operability. Proof: `fun a ha => inv_isOperational a ha`. One line, mathematical content trivial given the closure axiom.
- `OperationalField.toOperationalGroupUnits` — bridge instance from `OperationalField K` to `OperationalGroup K^×`. Proof: standard transfer of operational status from a field to its units.

The axiom profile of both objects is `[propext, Classical.choice, Quot.sound]` — the full ceiling. This is unexpected for one-line constructive proofs. v0.4.0 conducted a systematic reconnaissance to determine whether the `Classical.choice` dependency could be eliminated.

Reconnaissance — six test files. Six test files were constructed in v0.4.0 Stage 2 to probe the dependency. Each test file isolated a different hypothesis about the source of `Classical.choice`. The findings:

- Test 1: minimal-import file without `VRCycle/Algebra/Instances.lean` — still `[propext, Classical.choice, Quot.sound]`. Conclusion: the $\mathbb{Q}$ instance is not the source.
- Test 2: sorry-proof of the type signature alone — `[propext, sorryAx, Quot.sound]`. Conclusion: the type signature does not require `Classical.choice`; the dependency is in the proof.
- Tests 3–5: actual proof using `simp`, `rw`, or term-mode rewriting — all `[propext, Classical.choice, Quot.sound]`. Conclusion: every constructive proof path through `Units.val_inv_eq_inv_val` (the bridging lemma) pulls `Classical.choice`.
- Test 6: the lemma `Units.val_inv_eq_inv_val` itself — `[], axiom-free`. Conclusion: the lemma is constructive, but applying it in the field-level context pulls `Classical.choice`.

The diagnosis: the apparatus import chain `VRCycle.Apparatus.ModeA → VRCycle.Apparatus.Wrapping → VRCycle.Audit.Computable → Mathlib.Data.Real.Basic` brings `Mathlib.Data.Real.Basic` into the type-elaboration context. `Mathlib.Data.Real.Basic` introduces real-number instances that alter the resolution of `Inv K` for a generic `[Field K]`. The altered resolution pulls `Classical.choice` through the encoding of inverse on a field.

For `inv_isModeAOp_field`, the dependency is **import-structural**: the appearance of $(\cdot^{-1})$ in the theorem’s type signature triggers `Inv K` resolution under the altered imports, which brings `Classical.choice`.

For `OperationalField.toOperationalGroupUnits`, the dependency is **proof-structural**: the bridge instance must prove that the field’s operational status transfers to the units, and the proof connects `u.inv` to `u.val-1` (the field inverse), which brings `Classical.choice`.

Outcome C — structural requirement. The reconnaissance concluded that no isolation of these objects to a separate import context eliminates the dependency. The `Classical.choice` is structural to `mathlib`’s encoding of `Inv K`, not to the operational tracking discipline. Refactoring the apparatus import chain would not help; the dependency is in `mathlib`’s encoding, not in the cycle’s design.

Finding A16 records the outcome: A15 is structurally embedded, with two distinct mechanisms (import-structural and proof-structural). The cycle accepts the dependency as the honest cost of working with `mathlib`’s field-inverse encoding.

VII.4. Propagation through $\mathbb{Q}$ -Module objects

Finding A15 has a propagation consequence: any object that depends on `OperationalField.toOperationalGroupUnits` or `inv_isModeAOp_field` through bridge chains inherits the `Classical.choice` dependency.

In v1.0.0, the $\mathbb{Q}$ -Module instance and its supporting theorems exhibit this propagation. The bridge chain `OperationalField  $\mathbb{Q}$   $\rightarrow$  OperationalRing  $\mathbb{Q}$   $\rightarrow$  OperationalAddGroup  $\mathbb{Q}$` brings the field’s `Classical.choice` to the additive group level; from there, the module structure on $\mathbb{Q}$ inherits it. The axiom profiles:

- `instOperationalModuleRatRat : OperationalModule  $\mathbb{Q}$   $\mathbb{Q}$  — [propext, Classical.choice, Quot.sound].`
- `rat_smul_isOperational (general  $\mathbb{Q}$  closure theorem) — same profile.`
- `rat_half_smul_third_isOperational (concrete example) — same profile.`

The propagation is consistent with Finding A10 (carrier-type ceiling) when viewed at the carrier level: $\mathbb{Q}$ has ceiling `[propext, Classical.choice, Quot.sound]`, and every structure on $\mathbb{Q}$ has the same ceiling. The propagation source — Finding A15’s structural `Classical.choice` — is one of the ways the $\mathbb{Q}$ ceiling is established.

In contrast, $\mathbb{Z}$ -Module objects exhibit no such propagation. The $\mathbb{Z}$ ceiling is `[propext]` (no `Classical.choice`); the $\mathbb{Z}$ -Module instance is `[propext]`; no field-inverse reasoning is invoked in the $\mathbb{Z}$ chain.

VII.5. Honest constructivity framing

The cycle’s constructivity is now stated with precision:

VR-Algebra v1.0.0 is logically constructive with two documented structural exceptions.

The phrase “logically constructive” means: every mathematical claim of the cycle — every closure axiom, every Mode A theorem, every Mode B audit, every bridge instance — has a proof whose mathematical content is constructive. The closure axioms are stated without invoking classical machinery. The Mode B audit’s proof is one inference step over the apparatus, after constructive witness extraction from `mathlib`. The cycle does not, in principle, require `Classical.choice` to establish its mathematical content.

The phrase “two documented structural exceptions” refers to Finding A15: the two specific theorems whose axiom profiles register `Classical.choice` for reasons structural to `mathlib`’s encoding of `Inv K`, not for reasons of constructive failure in the underlying mathematics. These exceptions are honestly documented; they are not concealed and not redescribed. The cycle accepts the dependency as the cost of integration with `mathlib`’s encoding.

The phrase deliberately does not say “fully constructive”. An earlier formulation tested in the cycle’s documentation read “fully constructive”, but the systematic reconnaissance of Finding A15 disproved that claim. The honest statement reflects the actual dependencies of the cycle.

The methodological implication: the operational tracking discipline does not pretend to constructive purity it does not achieve. Where the encoding’s structure pulls `Classical.choice`, the axiom audit records this, and the cycle’s framing reflects this. The discipline is **operationally honest**: it tracks classical dependencies where they exist, accepting them as part of the encoding’s nature, and acknowledges them in the constructivity claim.

This honest framing distinguishes the present work from formalisations that obscure the boundary between constructive and classical content. The axiom audit is conducted for every public object; the result is reported; the boundary is the data of Finding A15 rather than a global claim.

VII.6. Summary of Part VII

Three findings document the axiom analysis of the cycle:

- **Finding A10** — ceiling = carrier-type property, not algebraic-depth property. Confirmed across four algebraic levels (group, ring, field, module) on $\mathbb{Z}$ and $\mathbb{Q}$.
- **Finding A15** — structural `Classical.choice` in field-inverse reasoning. Two objects (`inv_isModeAOp_field`, `OperationalField.toOperationalGroupUnits`) pull `Classical.choice` for reasons structural to `mathlib`’s encoding of `Inv K`.
- **Finding A16** — A15 outcome documented: structurally embedded, two distinct mechanisms (import-structural and proof-structural), no isolation refactoring eliminates the dependency.

The cycle’s constructivity is framed honestly: **logically constructive with two documented structural exceptions**. The framing reflects the actual dependencies registered in the axiom audit. The methodological position: operational tracking is **operationally honest**, recording classical dependencies where they exist rather than obscuring them.

Part VIII develops the Lean formalisation methodological observations — the complete catalog of twenty findings (A0–A19), categorised and cross-referenced, as the cycle’s full methodological signature in the Lean infrastructure.

Part VIII. Lean Formalisation Methodological Observations

VIII.0. Introduction

The previous parts developed the mathematical content of the cycle (Parts II–V) and the methodological observations specific to the construction process (Parts VI–VII). The present part collates the full catalog of twenty findings (A0–A19) accumulated through the cycle, organised by methodological category and cross-referenced to stage source.

This part is analogous to Part X of VR-Sets v1.0.1 and Part IX of VR-Forms v1.0.1, which document the methodological observations of their respective Lean formalisation cycles. Like those parts, the present one records what the formalisation revealed about the structural relationship between the apparatus framework and Lean’s typeclass infrastructure — observations that emerged from the formalisation work rather than from prior expectation.

Twenty findings are catalogued. They are organised into four thematic clusters:

- §VIII.1 — Recognition discipline observations (seven applications).
- §VIII.2 — Apparatus reuse observations (Finding A3 and its confirmations).

- §VIII.3 — Axiom hierarchy observations (Findings A1, A4, A5, A10, A14, A15, A16).
- §VIII.4 — Engineering observations (Findings A2, A7, A8, A11, A13, A18, A19).

§VIII.5 provides the full summary table linking each finding to stage source and category.

VIII.1. Recognition discipline observations

Seven findings catalog the recognition discipline applications discussed thematically in Part VI:

- **F11** (VR-Apparatus context). Removal of generic `Register` typeclass when no use case justified it. The inaugural application; precedes the present cycle.
- **A0** (v0.1.0). Omission of multiplicative `OperationalGroup` typeclass — no instances yet identified.
- **A6** (v0.1.0). Omission of bundled `OperationalAddSubgroup` structure — predicate suffices for use cases.
- **A9** (v0.2.0). Omission of `OperationalCommRing` — commutativity adds no operational content beyond ring.
- **Two Stage 5 v0.2.0 omissions**. Bridge theorems derivable from existing axioms by standard tactics; omitted as unnecessary.
- **A12** (v0.3.0). Introduction of multiplicative `OperationalGroup` when field-of-units construction provided the first instance. Closes the loop opened by A0.
- **A19** (v0.4.0). Plan-level correction of `IsModeA0p2` to unary `IsModeA0p` for fixed scalar when the architectural form was found type-incorrect during implementation.

The thematic significance — bidirectional discipline (removal and introduction) extended across levels (code and plans) — is developed in §VI.5.

VIII.2. Apparatus reuse observations

The principal finding of the cluster is Finding A3 in its general form, with its five confirmations through the cycle:

- **A3** (v0.1.0). Apparatus framework registers without modification for `OperationalAddGroup`. First confirmation.
- **A13** (v0.3.0). Apparatus framework registers without modification for `OperationalField`. Third confirmation in the cycle (after Ring in v0.2.0); the second confirmation through `MulGroup` is implicit in A12.
- **A17** (v0.4.0). Apparatus framework instance literally **reused** for `OperationalModule` — no new declaration needed. Fourth confirmation and structurally strongest.

The findings document the progressive strengthening of apparatus generality:

- Confirmation 1 (A3 itself): framework applies, new instance declared.
- Confirmations 2–4: same pattern across rings, multiplicative groups, fields.
- Confirmation 5 (A17): framework instance reused, no new declaration. The qualitative strengthening.

The thematic significance and the structural distinction between “structural extension” and “action extension” are developed in §III.3.

VIII.3. Axiom hierarchy observations

Seven findings document the axiom analysis of the cycle:

- **A1** (v0.1.0). Three-tier axiom hierarchy observed across `OperationalAddGroup` instances: `[]` (pure algebraic content), `[propext]` ($\mathbb{Z}$ -based), `[propext, Quot.sound]` ($\mathbb{Z}\text{Mod } n$ -based).
- **A4** (v0.2.0). $\mathbb{Z}$ axiom ceiling confirmed at `[propext]` for ring structure: integers carry no `Quot.sound` dependency.
- **A5** (v0.2.0). $\mathbb{Z}\text{Mod } n$ axiom ceiling confirmed at `[propext, Quot.sound]` for ring structure: quotient soundness enters with the quotient construction $\mathbb{Z} / n\mathbb{Z}$.
- **A10** (v0.2.0). Ceiling = carrier-type property, not algebraic-depth property. Initial formulation from observing `AddGroup` and `Ring` on the same carriers. Strengthened in v0.3.0 (`Field`) and v0.4.0 (`Module`).
- **A14** (v0.3.0). $\mathbb{Q}$ axiom ceiling for `Field` structure: `[propext, Classical.choice, Quot.sound]` — the full ceiling, propagating from `mathlib's Rat.instField`.
- **A15** (v0.3.0). Two specific theorems (`inv_isModeAOp_field`, `OperationalField.toOperationalGroupUnits`) pull `Classical.choice` for reasons structural to `mathlib's` encoding of `Inv K`. Initial observation.
- **A16** (v0.4.0). A15 outcome documented after systematic reconnaissance: two distinct mechanisms (import-structural for `inv_isModeAOp_field`, proof-structural for `toOperationalGroupUnits`), no isolation refactoring eliminates the dependency. Honest acceptance of structural dependency.

The thematic significance, detailed in Part VII, is the honest constructivity framing of the cycle: logically constructive with two documented structural exceptions, propagating predictably through bridge chains.

VIII.4. Engineering observations

Seven findings document engineering-level observations from the formalisation:

- **A2** (v0.1.0). Subgroup predicate sufficient without bundled structure. Mathematical content captured by `IsOperationalSubgroup S : Prop` directly; the bundled record `OperationalAddSubgroup G` was found unnecessary (also A6 in the recognition discipline cluster).
- **A7** (v0.2.0). Ring closure axiom structure: `one_isOperational` as constant axiom, `mul_isOperational` as binary closure axiom. Two axioms suffice; `npow_isOperational` derives by induction.
- **A8** (v0.2.0). Bridge instance from `OperationalRing` to `OperationalAddGroup` constructed via inferred ring lemmas; no manual transfer of additive closure axioms needed beyond predicate identification.
- **A11** (v0.2.0). Structural contrast between algebraic Mode B (image-of-subgroup, constructive witness via `mem_map`) and analytical Mode B (VR-Audit Hahn-Banach, classical existence selection through Zorn-equivalent). The contrast is structural, reflecting the underlying mathematics.
- **A13** (v0.3.0). Apparatus reuse confirmation for `Field` — already listed in §VIII.2 as part of the apparatus reuse cluster.
- **A18** (v0.4.0). `zsmul` vs `zpow` proof mechanics asymmetry: `zsmul_isOperational` requires explicit rewrite (`rw [natCast_zsmul]`) where `zpow_isOperational` admits bare exact. The asymmetry is in `mathlib's` `zsmul` versus `zpow` definitional unfolding; the mathematical content is the same.
- **A19** (v0.4.0). First heterogeneous binary operation in the VR cycle: scalar multiplication $\cdot : R \rightarrow M \rightarrow M$. The Mode A pattern was refined to unary for fixed scalar under implementation. Also listed in §VIII.1 as the plan-level recognition discipline application.

The thematic significance is local: each finding is a specific structural observation from the formalisation, contributing to the cumulative methodological signature without standalone narrative weight.

VIII.5. Findings summary table

The complete catalog of twenty findings, with stage source and category:

Finding	Stage	Category	Brief
F11	VR-Apparatus	Recognition discipline	Generic Register removed
A0	v0.1.0	Recognition discipline	OperationalGroup omitted (no instances)
A1	v0.1.0	Axiom hierarchy	Three-tier hierarchy on AddGroup
A2	v0.1.0	Engineering	Subgroup predicate suffices
A3	v0.1.0	Apparatus reuse	Framework applies to AddGroup
A4	v0.2.0	Axiom hierarchy	$\mathbb{Z}$ ceiling [propext] for Ring
A5	v0.2.0	Axiom hierarchy	ZMod n ceiling [propext, Quot.sound]
A6	v0.1.0	Recognition discipline	Bundled subgroup structure omitted
A7	v0.2.0	Engineering	Ring closure axiom structure
A8	v0.2.0	Engineering	Bridge instance construction
A9	v0.2.0	Recognition discipline	OperationalCommRing omitted
A10	v0.2.0+	Axiom hierarchy	Ceiling = carrier-type property
A11	v0.2.0	Engineering	Algebraic vs analytic Mode B structural contrast
A12	v0.3.0	Recognition discipline	OperationalGroup revived (loop closure)
A13	v0.3.0	Apparatus reuse	Framework applies to Field
A14	v0.3.0	Axiom hierarchy	$\mathbb{Q}$ ceiling for Field structure
A15	v0.3.0	Axiom hierarchy	Field-inverse Classical.choice
A16	v0.4.0	Axiom hierarchy	A15 structural outcome documented
A17	v0.4.0	Apparatus reuse	Module apparatus instance reused (strongest)
A18	v0.4.0	Engineering	zsmul/zpow proof mechanics asymmetry
A19	v0.4.0	Recognition discipline + Engineering	Heterogeneous binary operation refinement

(Two Stage 5 v0.2.0 omissions are tracked in the recognition discipline catalog but not individually numbered.)

VIII.6. Summary of Part VIII

Twenty findings constitute the cycle’s full methodological signature in Lean infrastructure. Organised by category:

- Seven recognition discipline applications (F11, A0, A6, A9, two Stage 5 v0.2.0 omissions, A12, A19).
- Four apparatus reuse confirmations (A3, A13, A17, plus the implicit v0.3.0 MulGroup case).
- Seven axiom hierarchy observations (A1, A4, A5, A10, A14, A15, A16).
- Seven engineering observations (A2, A7, A8, A11, A13, A18, A19).

(Findings A13 and A19 are cross-listed across categories.)

The methodological signature is intended to support extensions of the cycle: future work adding polynomial rings, matrix rings, nontrivial predicates, or additional substantive audits can be classified by the same scheme, with findings catalogued accordingly. The methodological discipline is preserved across extensions; the catalog is extensible.

Part IX develops the open questions and future directions sketched in Part I §I.5: polynomial rings, matrix rings, nontrivial operational predicates, additional substantive Mode B audits, and possible operational cycles in other domains.

Part IX. Open Questions and Future Directions

IX.0. Introduction

The cycle v1.0.0 closes the algebraic hierarchy core: groups, rings, fields, modules. The structural pattern is established; the apparatus generality is confirmed across five structures; the recognition discipline is observed bidirectionally. The methodological signature constitutes the cycle’s substantive contribution; the mathematical content (Mode A theorems, Mode B audit) demonstrates the apparatus on substantive content.

What remains open are extensions of the cycle to further algebraic content, to nontrivial operational predicates, to additional substantive Mode B audits, and to other mathematical domains. The present part sketches these directions without committing to specific timelines.

Contents: IX.1 — polynomial rings; IX.2 — matrix rings; IX.3 — nontrivial operational predicates; IX.4 — additional substantive Mode B audits; IX.5 — multiplicative group instances beyond fields; IX.6 — non-commutative algebra; IX.7 — operational topology and category theory.

IX.1. Polynomial rings

The construction `OperationalRing (Polynomial R)` for `[OperationalRing R]` extends the apparatus to polynomial structures. Mathematically: a polynomial with operational coefficients should be operational; addition, multiplication, scalar multiplication of polynomials should preserve operability.

The construction is anticipated but not included in v1.0.0. The reason is scope discipline: the algebraic hierarchy core is itself substantial, and adding polynomial rings would shift the cycle’s focus from “hierarchy

demonstration” to “hierarchy plus extensions”. The polynomial extension is natural for v1.1.0+ when the hierarchy core is stable.

Technical considerations:

- `mathlib`’s `Polynomial R` requires `[Semiring R]`. The bridge `[Ring R] → [Semiring R]` is automatic; no obstacle.
- The operational closure axioms transfer naturally: polynomial addition is coefficient-wise; polynomial multiplication is the convolution; both preserve operability if coefficients are operational.
- The predicate `IsOperational (p : Polynomial R)` could be defined as “every coefficient of `p` is operational”. For the trivial-predicate case (current instances), this reduces to `True`.

The polynomial extension is the natural next step structurally. It does not introduce new methodological content beyond the apparatus reuse pattern already established (Finding A3) and the predicate-handling pattern (§II.6).

IX.2. Matrix rings

The construction `OperationalRing (Matrix n n R)` for `[OperationalRing R]` and `n : Type*` (the index type for the matrix) extends to matrix structures.

Technical considerations:

- `mathlib`’s `Matrix n n R` has a ring structure when `R` is a ring and `n` is finite. The bridge to the operational ring layer is parallel to the polynomial case.
- The operational closure transfers similarly: matrix addition and multiplication preserve operability if entries are operational.
- The predicate `IsOperational (M : Matrix n n R)` analogous: “every entry is operational”.

The methodological observation: polynomial rings and matrix rings are both **functorial** constructions over the operational ring layer. A general functorial framework — “if `F` is a functor that takes a ring to a ring, and `F` admits operational lifting, then `F R` is an operational ring whenever `R` is” — could subsume both cases. The general framework is not in the present cycle’s scope but is the natural abstraction direction.

IX.3. Nontrivial operational predicates

In v1.0.0, every operational typeclass instance uses `IsOperational := fun _ => True`. The trivial predicate is consistent with the position established in `VR-Numbers` (integer and rational arithmetic is operational throughout), but it does not exercise the framework’s capacity to distinguish operational from non-operational elements.

Three candidate domains for the first nontrivial operational predicate:

Computable functions ring. The set of computable functions $\mathbb{R} \rightarrow \mathbb{R}$ forms a ring (under pointwise addition and multiplication). The operational predicate distinguishes computable from non-computable functions; closure under ring operations is non-trivial (the pointwise sum and product of computable functions are computable).

This is the most natural domain for connecting `VR-Algebra` to `VR-Audit`: computable functions are the analytical objects of `VR-Audit`, now viewed as algebraic objects (a ring). A future cycle could develop `OperationalRing (ComputableFunction  $\mathbb{R}$   $\mathbb{R}$ )` with `IsOperational := IsComputable`.

Algebraic integers. The ring of algebraic integers — complex numbers that are roots of monic polynomials with integer coefficients — is a subring of $\mathbb{C}$. The operational predicate distinguishes algebraic integers from non-algebraic complex numbers (e.g., e or π are not algebraic). Closure under ring operations is non-trivial (sum and product of algebraic integers are algebraic integers; this is a substantive theorem of number theory).

A future cycle could develop `OperationalRing (AlgebraicInteger)` with `IsOperational := IsAlgebraicInteger`. The cycle would exercise the apparatus on a substantive number-theoretic content.

Decidable subgroup membership. For a group G , a subgroup $H \subseteq G$ may have decidable or undecidable membership. The operational predicate on subgroups distinguishes decidable-membership subgroups from those whose membership is undecidable.

This domain connects to computational group theory. The cycle would develop `OperationalAddSubgroup` (or its multiplicative analogue) with `IsOperational := DecidableMembership`.

Each of the three domains is a substantial cycle in itself. The selection depends on which connects most naturally to existing infrastructure and to the methodological direction of the larger programme.

IX.4. Additional substantive Mode B audits

The cycle conducts one substantive Mode B audit (image-of-subgroup, Part V). Additional substantive audits await development:

Lagrange’s theorem. For a finite group G and a subgroup H , the order of H divides the order of G . Operationally: for an operational group with decidable equality and decidable subgroup membership, the order divides relation is decidable, and the audit tracks operational content of the divisibility.

Sylow theorems. For a finite group G and prime p , Sylow subgroups exist and are conjugate. Operationally: when the group has decidable structure, Sylow subgroup construction can be made operational.

Fundamental theorem of finitely generated abelian groups. Every finitely generated abelian group decomposes uniquely as a product of cyclic groups. Operationally: the decomposition is computable when the group has operational structure (e.g., a finitely generated abelian group given by generators and relations).

Each audit would document the operational content of a classical algebraic theorem, paralleling the role of Hahn-Banach in VR-Audit. The audits are substantial standalone investigations.

IX.5. Multiplicative group instances beyond fields

The multiplicative `OperationalGroup` typeclass has one substantive instance in v1.0.0: the units of an operational field. Other potential sources of multiplicative group instances:

- **Galois groups of finite extensions.** The Galois group of a separable polynomial over a field is a finite group; operationally, when the polynomial is computable, the Galois group is operationally accessible.
- **Symmetric groups.** S_n is operational for finite n ; the operational predicate is trivial (True), but the structural integration with `mathlib`’s permutation theory is non-trivial.
- **Linear groups over operational rings.** $GL(n, R)$ for an operational ring R and finite n admits an operational group structure when matrix invertibility is decidable.

Each instance extends the multiplicative group apparatus to a new mathematical context. The instances are independent of one another and can be developed as standalone extensions.

IX.6. Non-commutative algebra

The cycle v1.0.0 considers rings without commutativity restrictions (the operational ring typeclass is layered over `mathlib`'s `Ring`, which does not require commutativity). However, all instances in v1.0.0 are on commutative carriers ($\mathbb{Z}$, $\mathbb{ZMod}\ n$, $\mathbb{Q}$).

Future cycles could develop substantive non-commutative instances:

- Matrix rings over commutative rings (non-commutative when $n \geq 2$).
- Group rings of finite groups over commutative rings.
- Quaternion algebras over operational fields.
- Endomorphism rings of finitely generated modules.

The framework supports the extensions structurally; the methodological observation about non-commutativity — that the apparatus does not distinguish commutative from non-commutative content at the operational level — is a worthwhile finding for future cycles.

IX.7. Other mathematical domains

Beyond algebra, the apparatus framework should apply to topology, category theory, and other mathematical content. The investigations are parallel to the present cycle but on different content:

Operational topology. The apparatus on topological structures — operational topological spaces, operational continuous functions, operational compactness and connectedness. The structural framework is the same; the closure axioms differ (topological operations: union of open sets, finite intersection, etc.).

Operational category theory. The apparatus on categorical structures — operational categories, operational functors, operational natural transformations. The framework adapts; the apparatus typeclass instance pattern transfers.

Operational measure theory. The apparatus on measure-theoretic structures — operational σ -algebras, operational measures, operational measurable functions. This connects naturally to VR-Audit (analysis) but at a different level of mathematical structure.

Each direction is a separate cycle, paralleling VR-Algebra structurally. The cumulative effect would be to demonstrate the apparatus framework's generality across mathematical domains — a programme-level claim to be supported by domain-level demonstrations.

IX.8. Summary

The cycle v1.0.0 closes the algebraic hierarchy core. The directions sketched in this part — polynomial rings, matrix rings, nontrivial predicates, additional Mode B audits, non-commutative instances, other domains — are extensions that can be developed independently, each contributing to the larger demonstration of the apparatus framework's generality.

The methodological signature established in the present cycle (Parts VI–VIII) supports extensions in the same form: each new content domain or instance is developed by the same architectural pattern, with findings catalogued by the same scheme. The cycle's contribution to the larger VR programme is thus twofold: (i) the algebraic content itself, demonstrating apparatus generality on a second mathematical domain (after analysis in VR-Audit); (ii) the methodological discipline, available to structure subsequent cycles in the programme.

References

- Aluffi, P. (2009). *Algebra: Chapter 0*. Graduate Studies in Mathematics, Vol. 104. American Mathematical Society.
- Avigad, J. (2022). The Mechanization of Mathematics. *Notices of the American Mathematical Society*, 69(5), 681–689.
- Bishop, E. (1967). *Foundations of Constructive Analysis*. McGraw-Hill.
- Bridges, D., & Richman, F. (1987). *Varieties of Constructive Mathematics*. Cambridge University Press.
- de Moura, L., & Ullrich, S. (2021). The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction — CADE 28* (pp. 625–635). Springer.
- Hungerford, T. W. (1974). *Algebra*. Graduate Texts in Mathematics, Vol. 73. Springer.
- Lang, S. (2002). *Algebra* (3rd ed.). Graduate Texts in Mathematics, Vol. 211. Springer.
- mathlib Community. (2020). The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)* (pp. 367–381). ACM.
- Mac Lane, S., & Birkhoff, G. (1999). *Algebra* (3rd ed.). AMS Chelsea Publishing.
- Reznik, V. (2026). *VR. A Formal System: A Minimalist Axiomatization of Arithmetic Grounded in Leibnizian Void*. Zenodo. DOI: 10.5281/zenodo.20212092
- Reznik, V. (2026). *VR-Numbers: Operational Extensions over the Natural Numbers of VR*. Zenodo. DOI: 10.5281/zenodo.20272743
- Reznik, V. (2026). *VR-Sets: An Operational Theory of Sets Grounded in Leibnizian Void*. Zenodo. DOI: 10.5281/zenodo.20303536
- Reznik, V. (2026). *VR-Forms: An Operational Two-Register Apparatus over VR-Sets*. Zenodo. DOI: 10.5281/zenodo.20355939
- Reznik, V. (2026). *VR-Audit: Operational Reformulation of the Hahn-Banach Extension Theorem*. Zenodo. DOI: 10.5281/zenodo.20364111
- Reznik, V. (2026). *VR-Sets-ZFA: A Coinductive Parallel for Non-Well-Founded Operational Sets*. Zenodo. DOI: 10.5281/zenodo.20369346
- Reznik, V. (2026). *VR-Apparatus: An Abstract Formalisation of the Operational Tracking Discipline*. Zenodo. DOI: 10.5281/zenodo.20381417
- Reznik, V. (2026). *VR-Algebra Lean Formalisation* (Software, Git tag v1.12-vr-operational-algebra-v1.0.0, commit 5e8f8e0). GitHub: <https://github.com/inventor1975/VRCycle>
- Simpson, S. G. (2009). *Subsystems of Second Order Arithmetic* (2nd ed.). Cambridge University Press.
- Weihrauch, K. (2000). *Computable Analysis: An Introduction*. Springer-Verlag.
-

Acknowledgement of AI assistance

The formal exposition of this preprint was prepared with the assistance of Claude (Anthropic, model Opus 4.7) for drafting, English translation, language polishing, and consistency checking, in the same configuration as the earlier works of the VR cycle.

The Lean 4 formalisation underlying this preprint was carried out using a parent-child review architecture: Claude Opus 4.7, holding the architectural and methodological role, made all stage-level design decisions and reviewed each stage's output; Claude Sonnet 4.6, running in Claude Code, wrote the Lean code, ran `lake build`, and reported `#print` axioms for every public object. The cycle followed the protocol of plan-before-code with tested prototype, acceptance criteria, and axiom audit. The same architecture was used in the preceding Lean cycles of VR-Sets, VR-Forms, and VR-Apparatus.

All mathematical content, definitions, theorems, ontological positions, methodological decisions, and architectural authority are due to the author.